

OpenShift





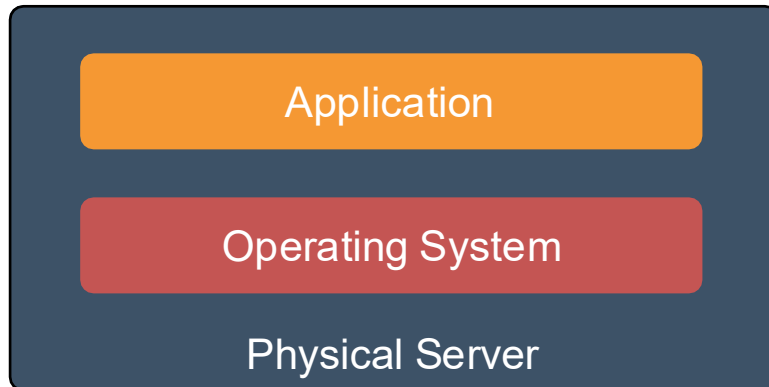
Data Center Evolution



Monolithic

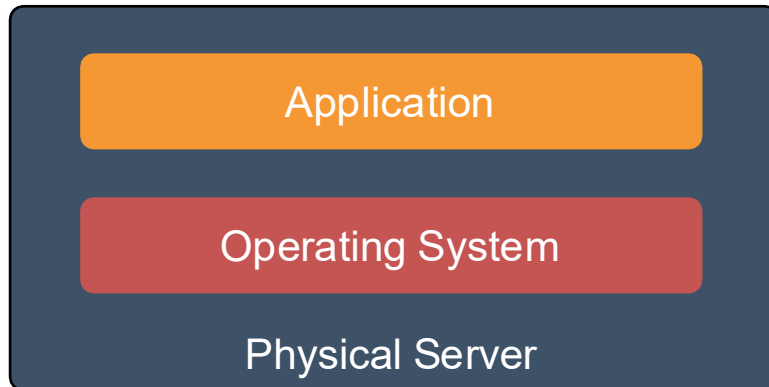


Monolithic Server Architecture



One physical server, one application

Monolithic Server Architecture



One physical server, one application

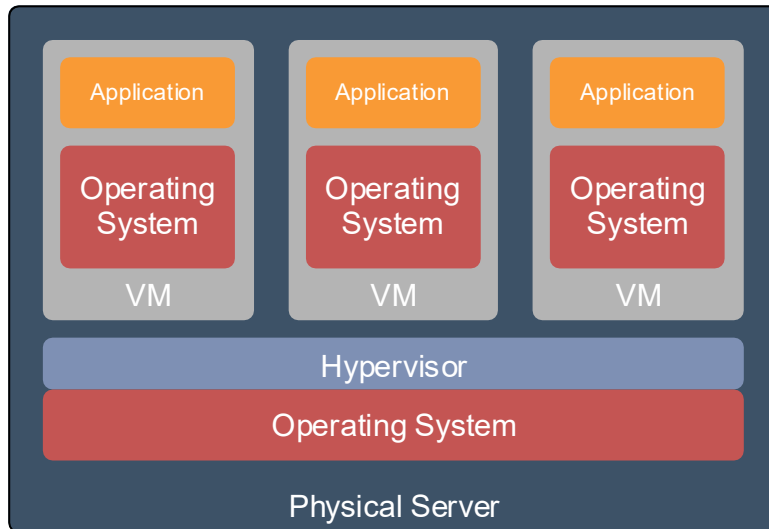
Problems

- Slow deployment times
- Cost
- Wasted resources
- Difficult to scale
- Difficult to migrate

Virtualized



Virtualized Infrastructure

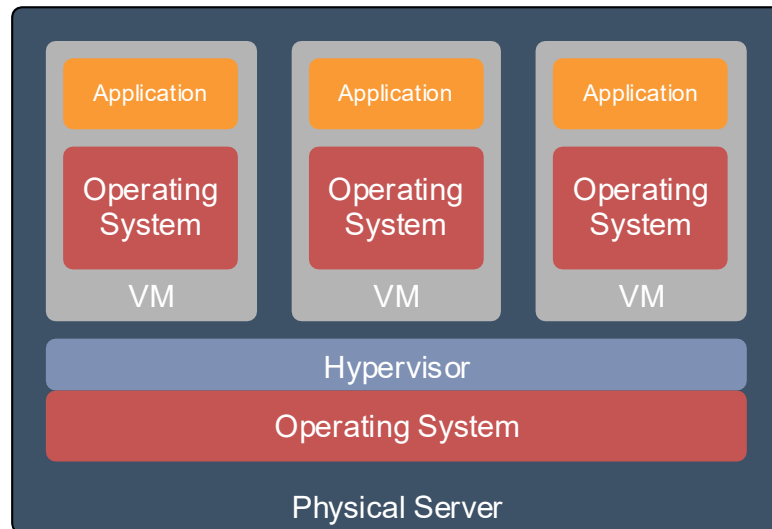


One physical server, multiple applications

Discussion

What are some of the advantages and disadvantages of Virtual Machines?

Virtualized Infrastructure - Advantages

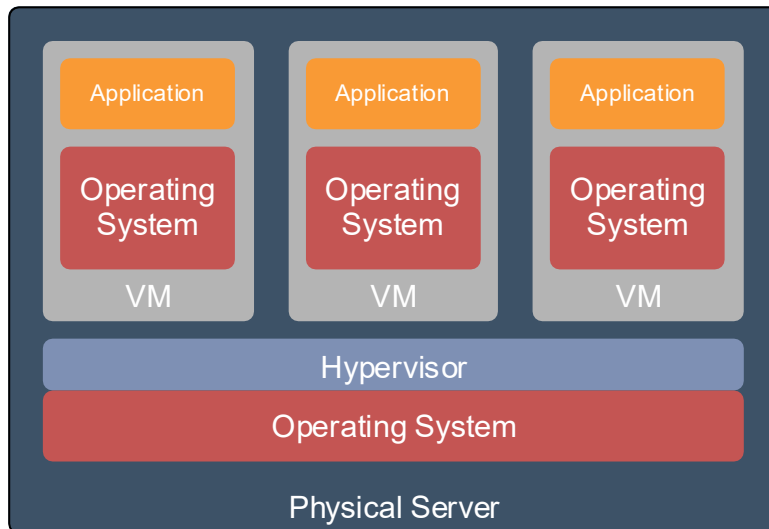
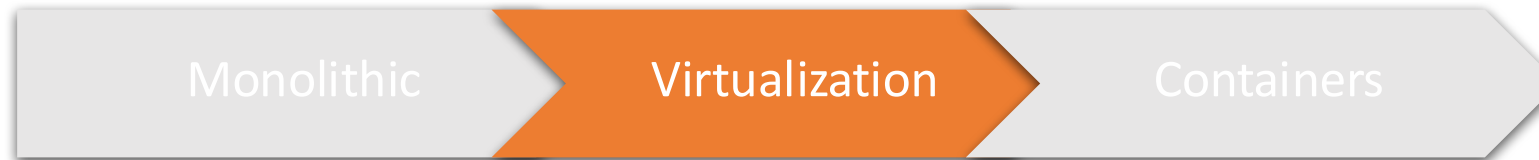


One physical server, multiple applications

Advantages

- Better resource pooling
- Easier to Scale
- Enables Cloud/IaaS
 - Rapid elasticity
 - Pay as you go model

Virtualized Infrastructure - Limitations



One physical server, multiple applications

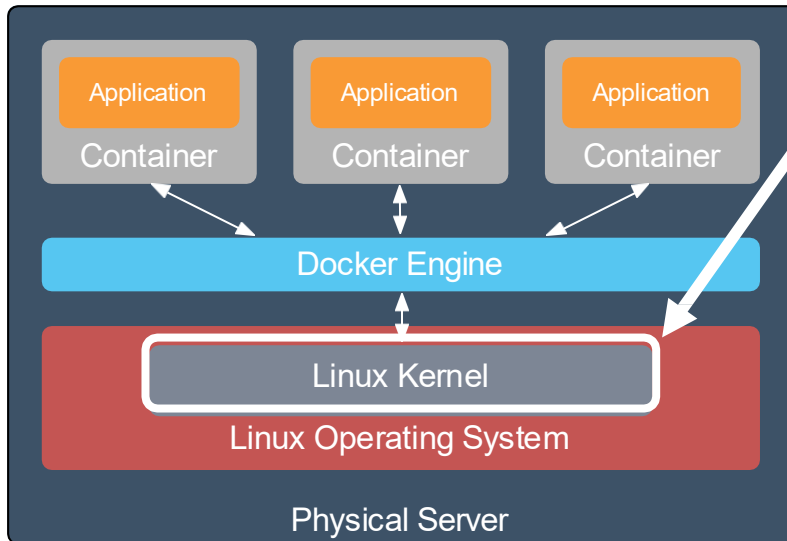
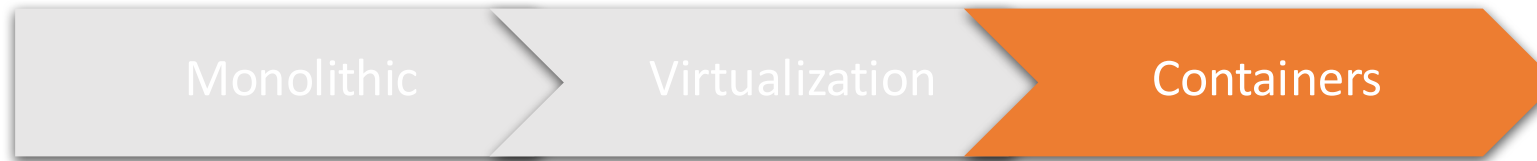
Limitations

- Each VM requires:
 - CPU allocation
 - Storage
 - RAM
 - Guest Operating System
- More VMs, more wasted resources
- Application portability not guaranteed

Containers

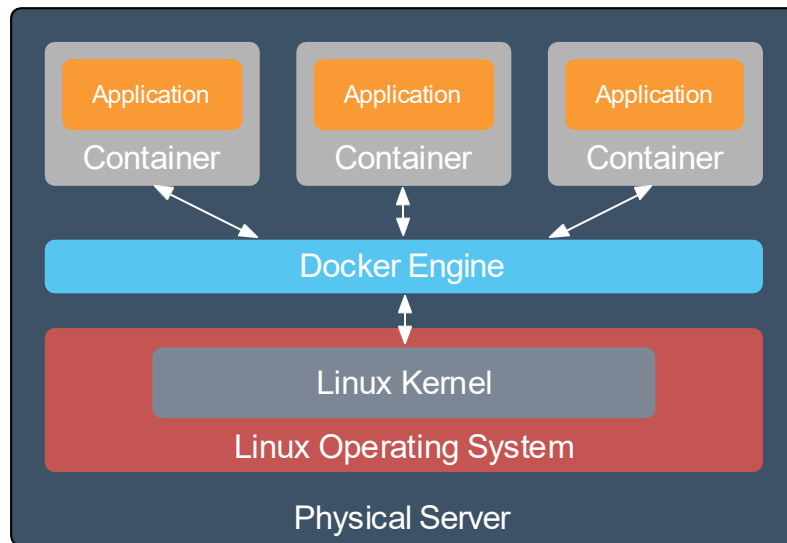
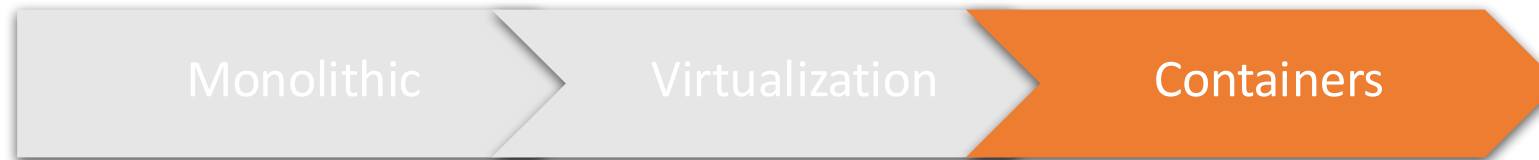


Containers



Shared kernel on the host to run multiple guest applications

Containers - Advantages

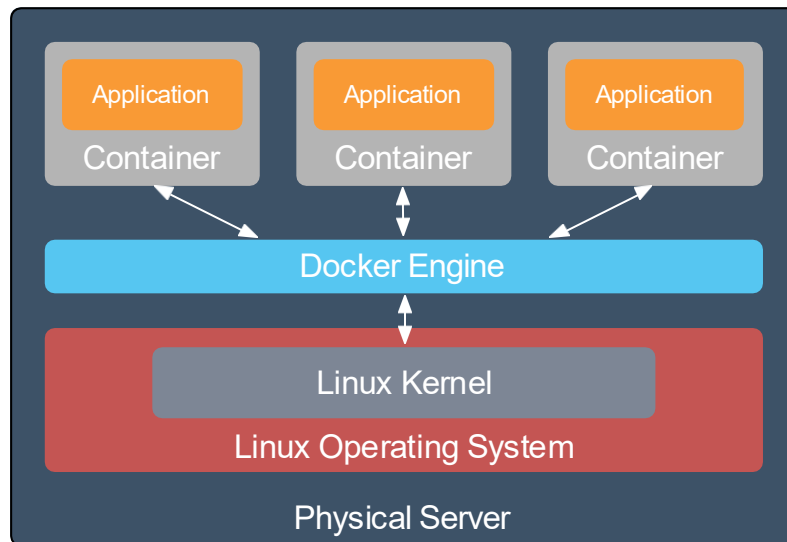
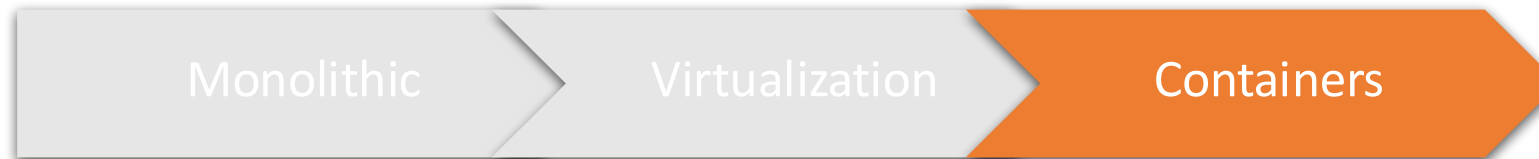


Shared kernel on the host to run multiple guest applications

Advantages over VMs

- Containers are more lightweight
- No need to install a guest Operating System
- Less CPU, RAM, storage overhead
- More containers per machine
- Greater portability

Containers - Challenges



Shared kernel on the host to run multiple guest applications

Container Challenges

- Early Docker focused on single-node operations
- Up to user to cluster Docker hosts and manage deployment of containers on cluster
- User solves for automatic scale out of applications
- User solves for service discovery between application components (microservices)

Container – Concept of Operations



Container based virtualization

Uses the kernel on the host operating system to run multiple guest instances

- Each guest instance is a container
- Each container has its own

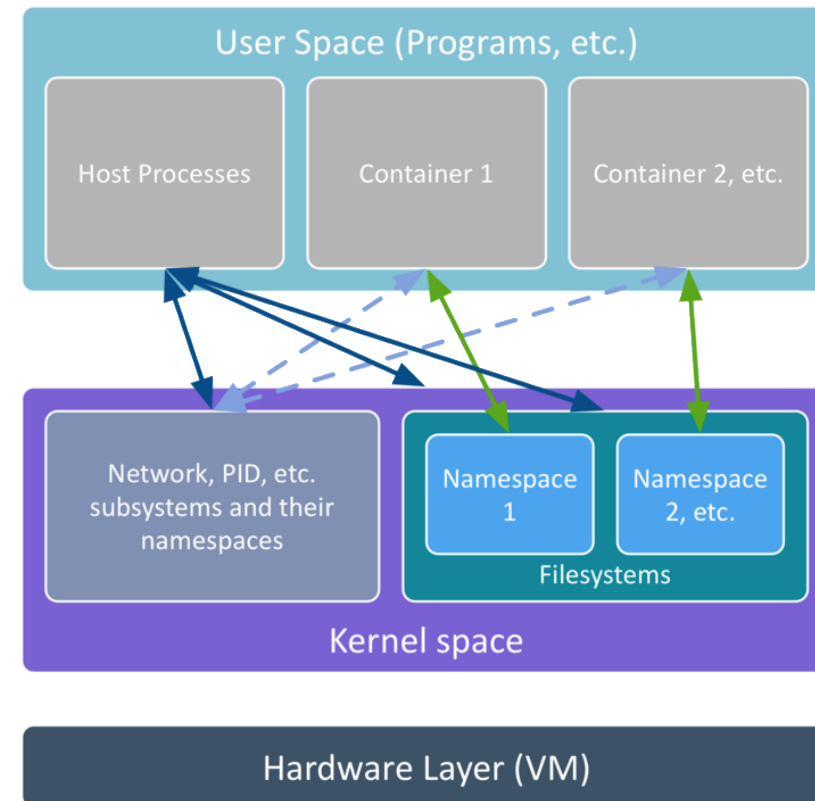
- Root filesystem
- Processes
- Memory
- Devices
- Network Ports



Isolation with Namespaces

Namespaces - Limits what a container can see (and therefore use)

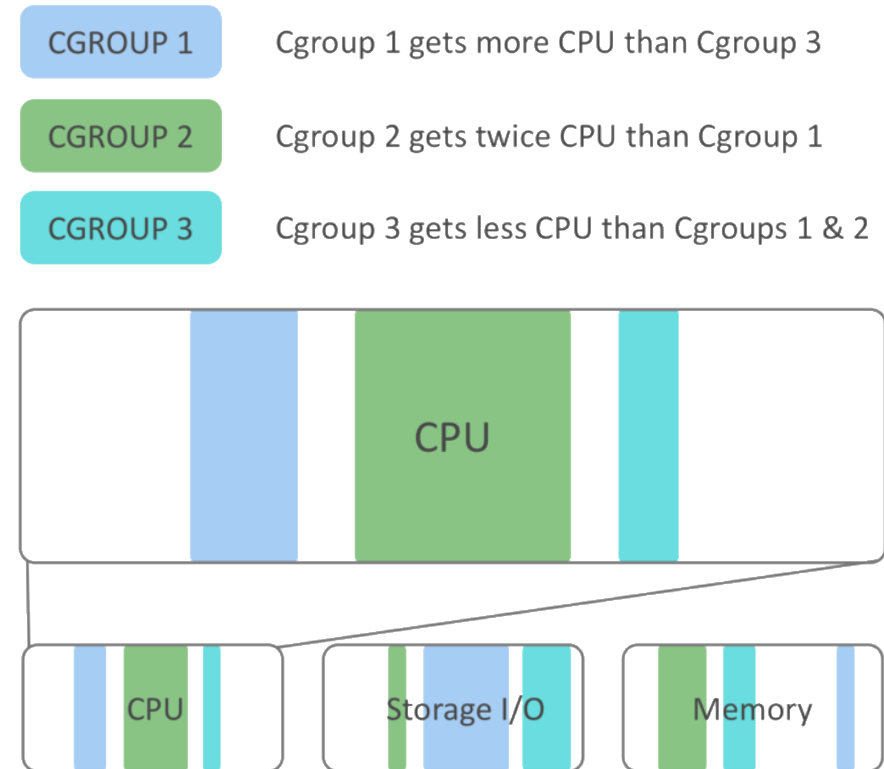
- Namespace wrap a global system resource in an abstraction layer
- Processes running in that namespace think they have their own, isolated resource
- Isolation includes:
 - Network stack
 - Process space
 - Filesystem mount points
 - etc.



Isolation with Control group (Cgroups)

Cgroups - Limits what a container can use

- Resource metering and limiting
 - CPU
 - MEM
 - Block/IO
 - Network
- Device node (/dev/*) access control



Container Use Cases



DevOps



Developers

Focus on applications inside the container



Operations

Focus on orchestrating and maintaining
containers in production

Container use-cases

Development

- Allows the ability to define the entire project configuration and tear-down/recreate it easily
- Supports multiple versions of application simultaneously

Container use-cases

Development

- Allows the ability to define the entire project configuration and tear-down/recreate it easily
- Supports multiple versions of application simultaneously

Test Environments

- Same container that developers run is the container that runs in test lab and production – includes all dependencies
- Well formed API allows for automated building and testing of new containers

Container use-cases

Development

- Allows the ability to define the entire project configuration and tear-down/recreate it easily
- Supports multiple versions of application simultaneously

Test Environments

- Same container that developers run is the container that runs in test lab and production – includes all dependencies
- Well formed API allows for automated building and testing of new containers

Microservices

- Design applications as suites of services, each written in the best language for the task
- Better resource allocation
- One container per microservice vs. one VM per microservice
- Can define all interdependencies of services with templates

Questions



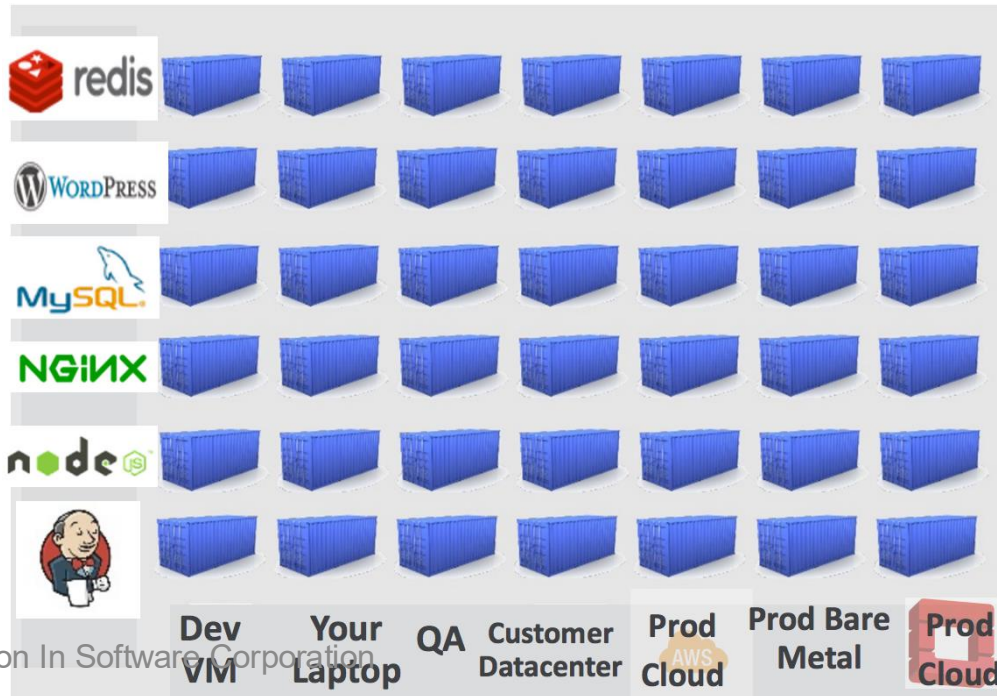
Docker Overview



What is Docker?

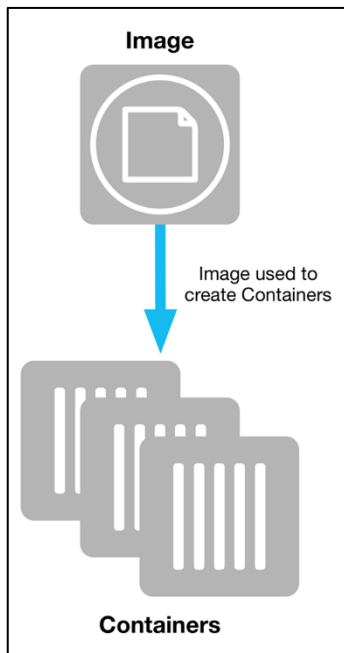


Docker allows you to package an application with all of its dependencies into a standardized unit for software development.



Terminology

Image



Read only template used to create containers

Built by you or other Docker users

Stored in Docker Hub, Docker Trusted Registry or your own Registry

Terminology

Image

Read only template used to create containers

Built by you or other Docker users

Stored in Docker Hub, Docker Trusted Registry or your own Registry

Container

- Isolated application platform
- Contains everything needed to run your application
- Based on one or more images

The Docker Platform Components



Docker Engine

Docker Engine

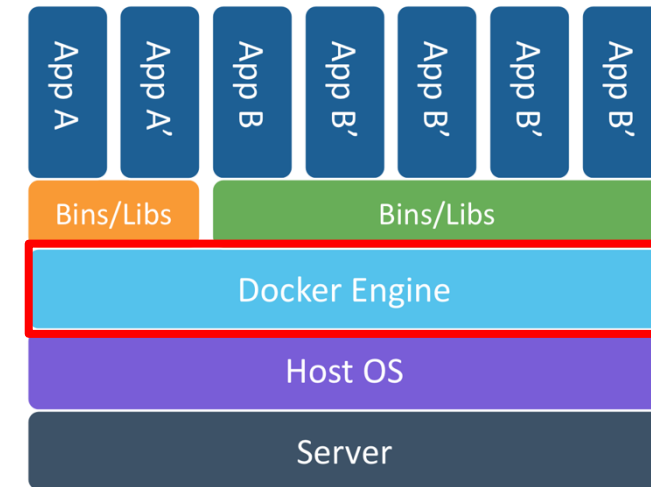
Docker Registry

Docker Compose

Docker Swarm

*Lightweight runtime program to **build, ship, and run Docker containers***

- Also known as **Docker Daemon**
- Uses Linux Kernel namespaces and control groups
 - **Linux Kernel (>= 3.10)**
- Namespaces provide an isolated workspace



Docker Engine

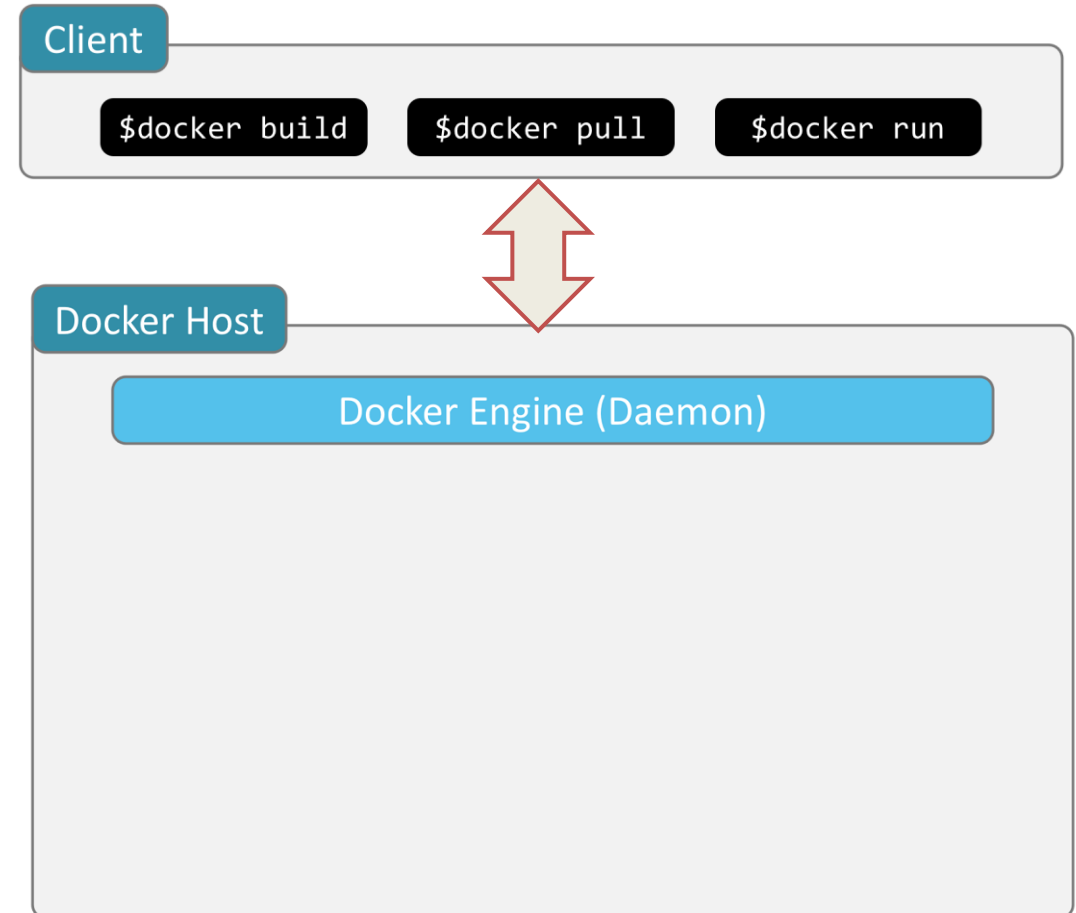
Docker Engine

Docker Registry

Docker Compose

Docker Swarm

- The Docker Client is the `docker` binary
 - Primary interface to the Docker Host
- Accepts commands and communicates with the Docker Engine (Daemon)



Docker Engine

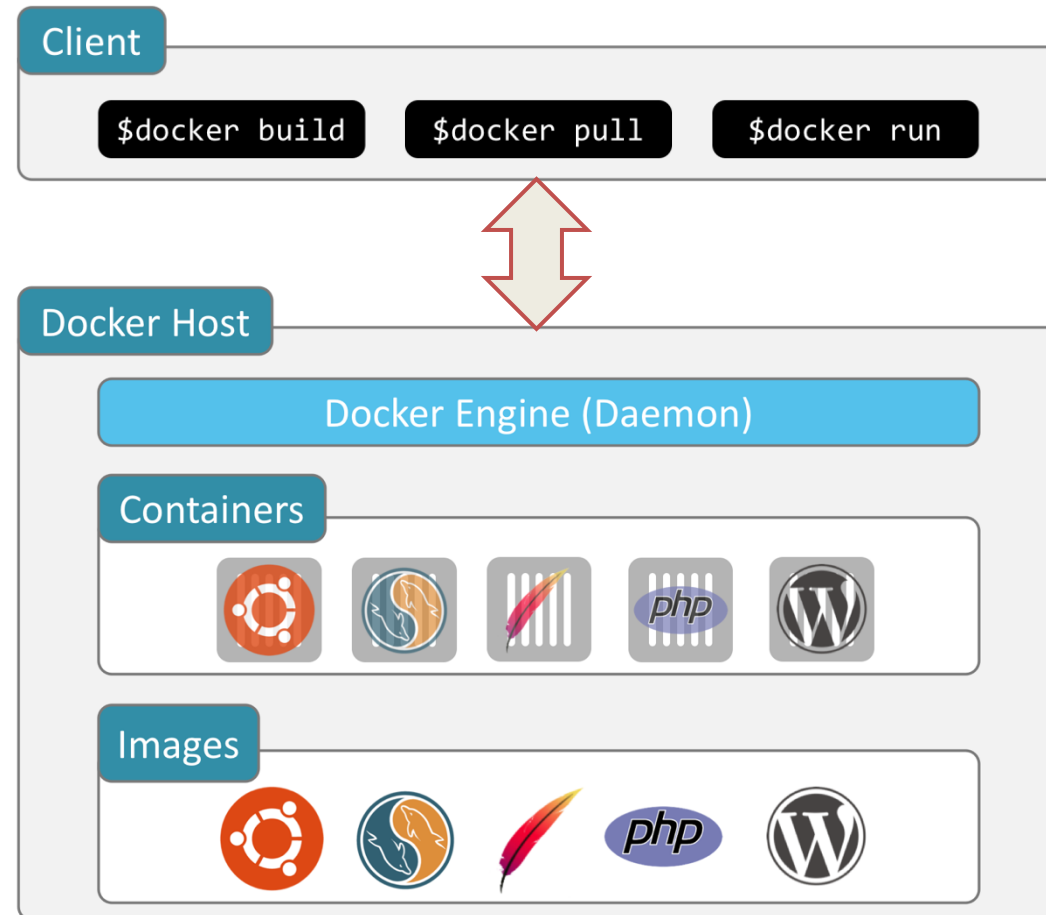
Docker Engine

Docker Registry

Docker Compose

Docker Swarm

- Lives on a Docker host
- Creates and manages containers on the host



Docker Registry

Docker Engine

Docker Registry

Docker Compose

Docker Swarm

Image Storage & Retrieval System



- Docker Engine **Pushes** Images to a Registry
- Version Control
- Docker Engine **Pulls** Images to Run



Docker Registry

Docker Engine

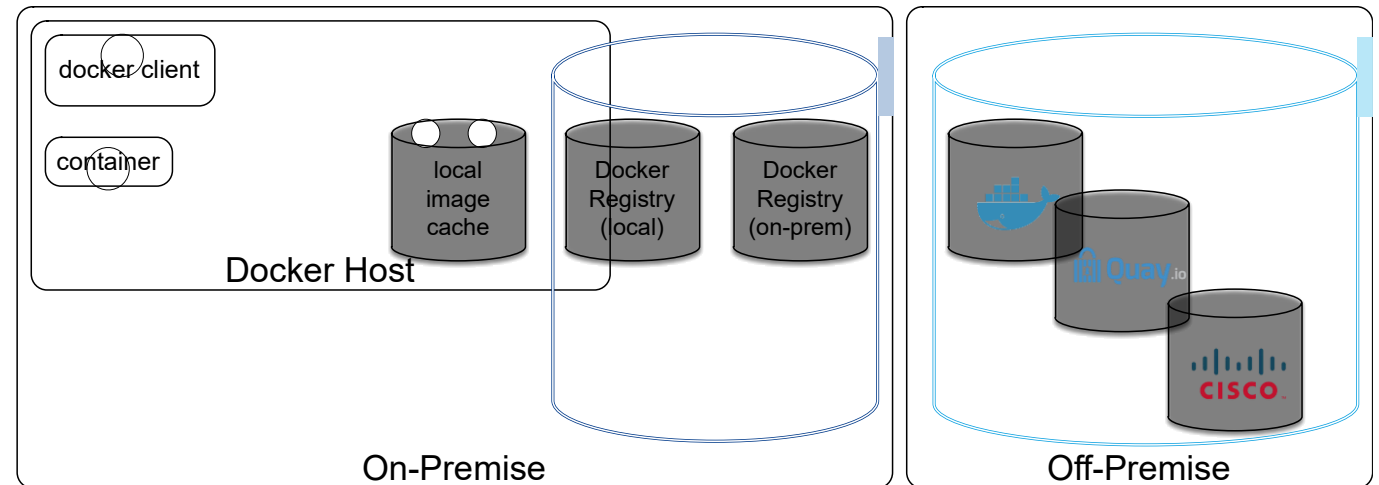
Docker Registry

Docker Compose

Docker Swarm

Types of Docker Registries

- **Local Docker Registry**
(On Docker Host)
- **Remote Docker Registry**
(On-Premise/Off-Premise)
- **Docker Hub**
(Off-Premise)



Docker Engine and Registry

Docker Engine

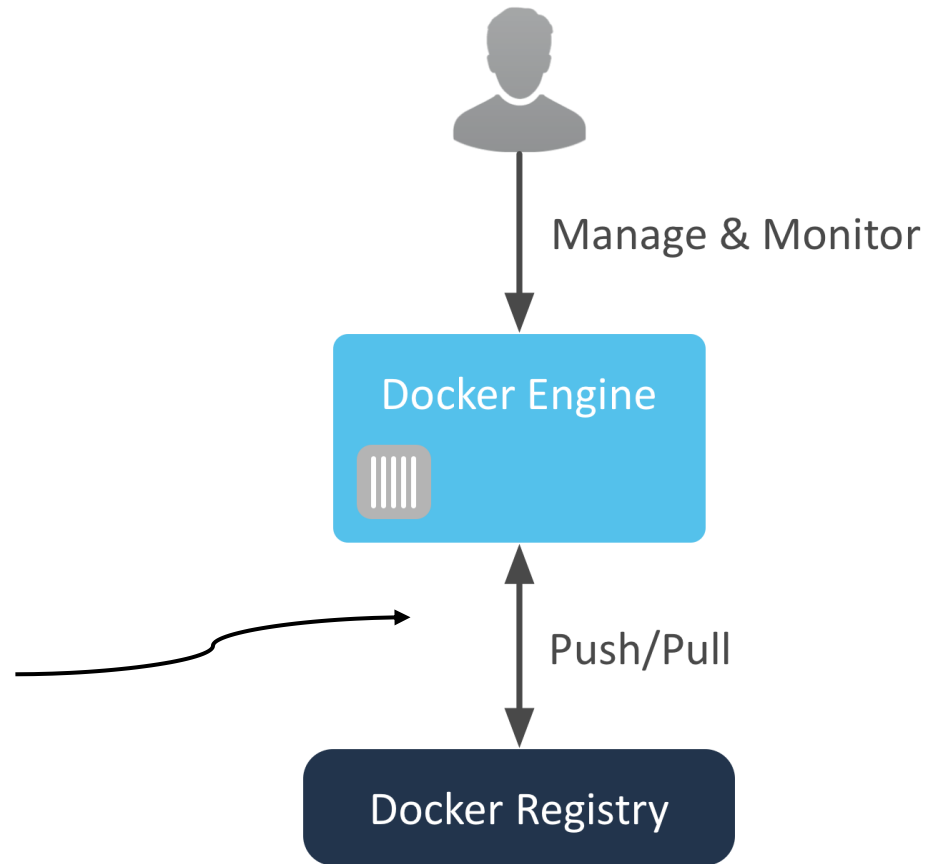
Docker Registry

Docker Compose

Docker Swarm

Docker Engine:

- **Pushes** Images to a Registry
- **Pulls** Images to Run



Docker Registry

Docker Engine

Docker Registry

Docker Compose

Docker Swarm

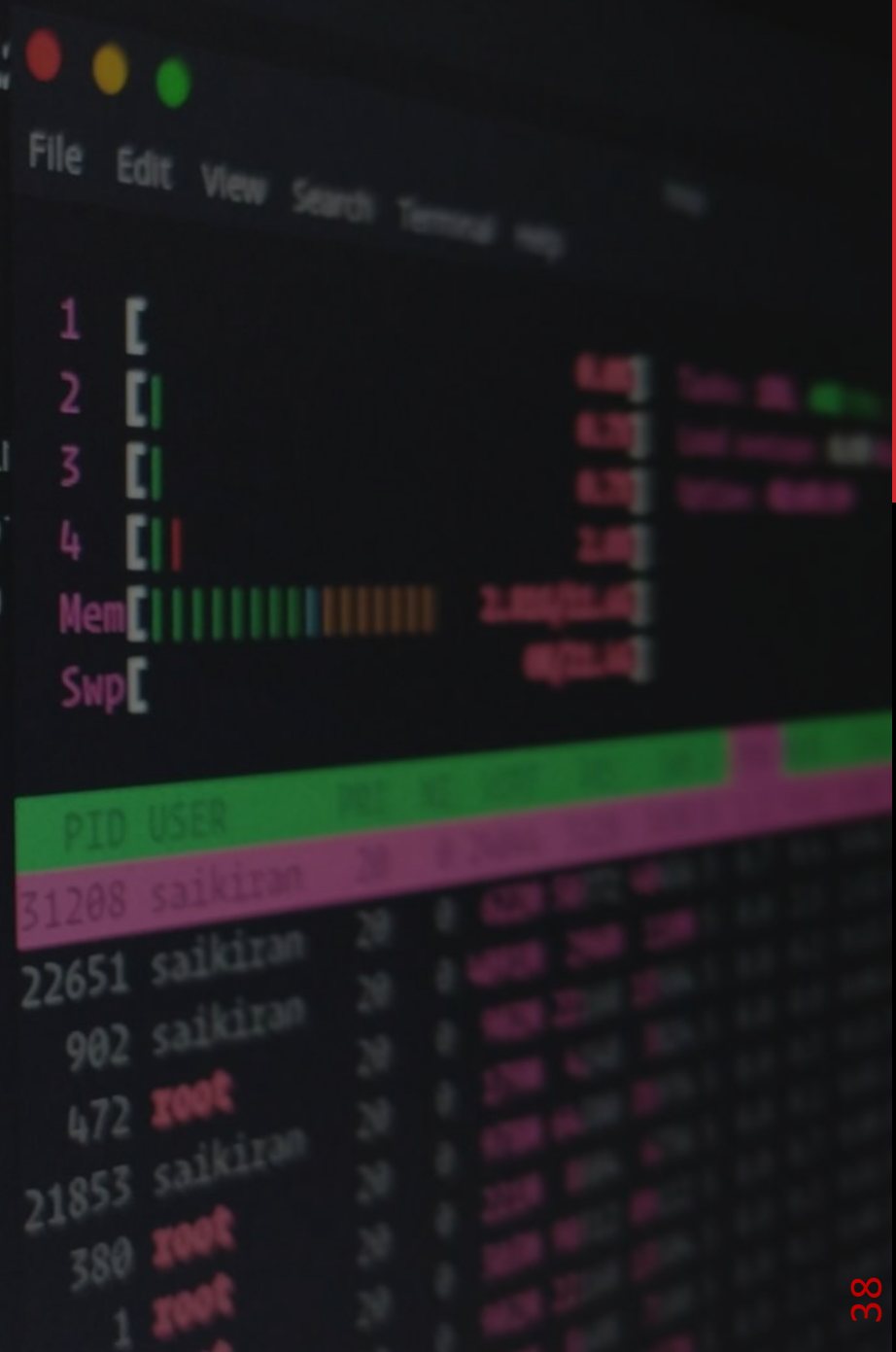
The registry and engine both present APIs

- All of Docker's functionality will utilize these APIs
- RESTFUL API
- Commands presented with Docker's CLI tools can also be used with curl and other tools

Labs: Openshift Resources



OpenShift Architecture



OpenShift Features

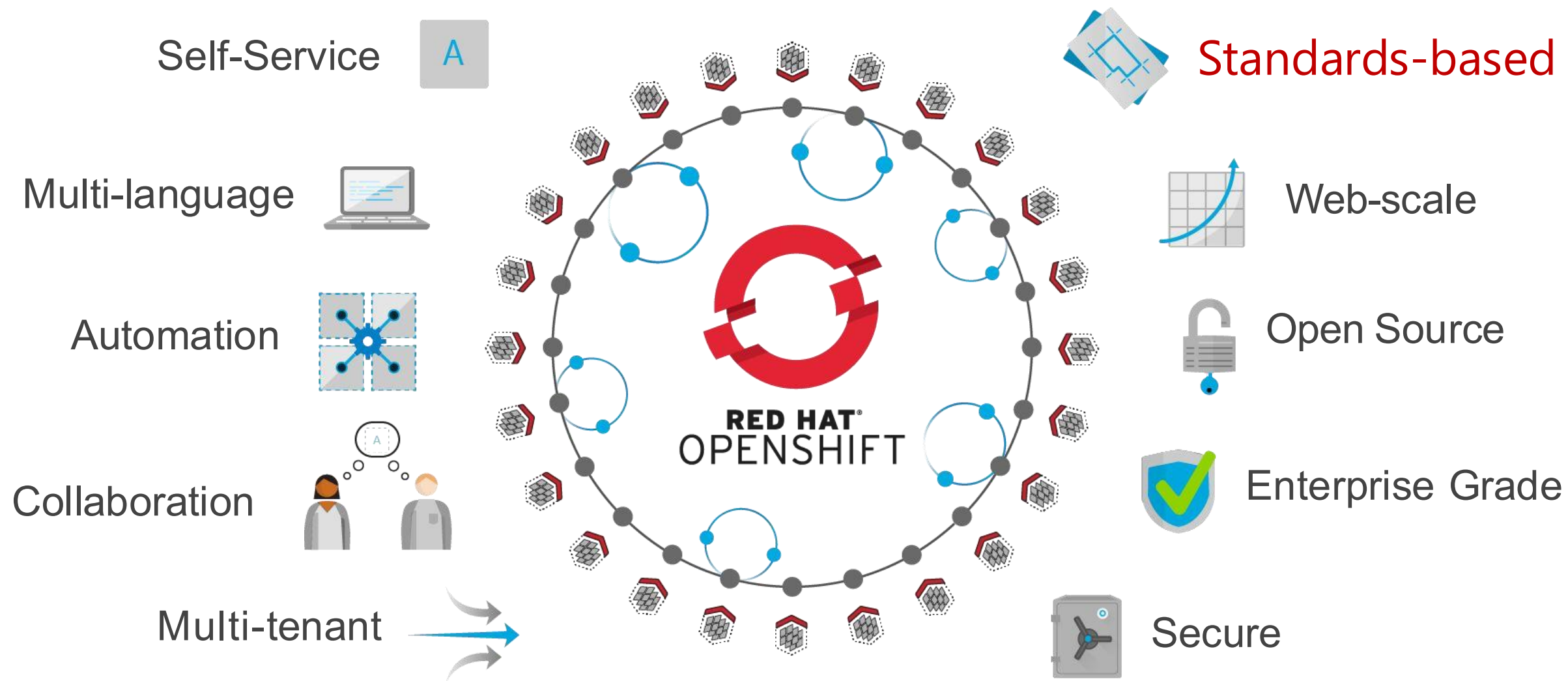


	Kubernetes	OpenShift
Multi-container, multi-host scheduling	✓	✓
Self-service provisioning	✓	✓
Service discovery	✓	✓
Persistent Storage	✓	✓
Multi-tenancy		✓
Networking (SDN)		✓
Image Registry		✓
Image Build Tools		✓
Metrics		✓
Log Aggregation		✓
Ingress		✓
UX: Console, Service Catalog, "oc"		✓
Secured by Default		✓



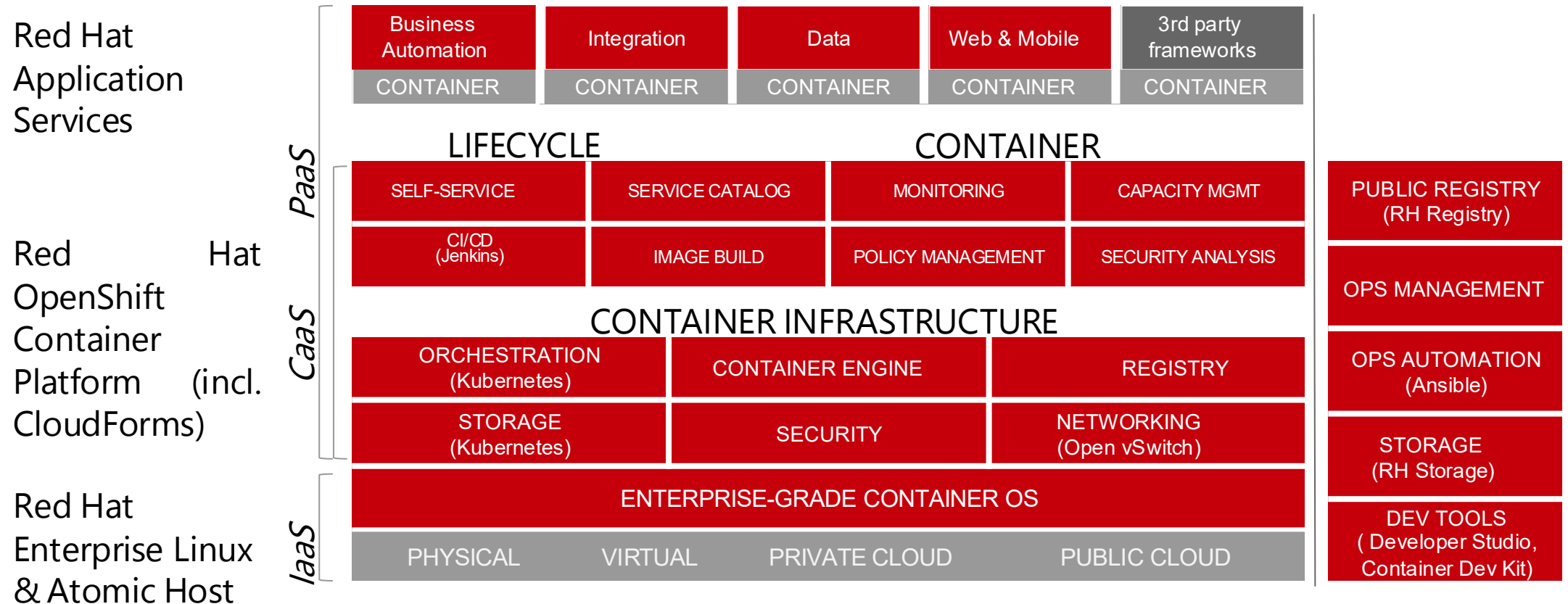
“Kubernetes with
the batteries
included!”

OpenShift Features

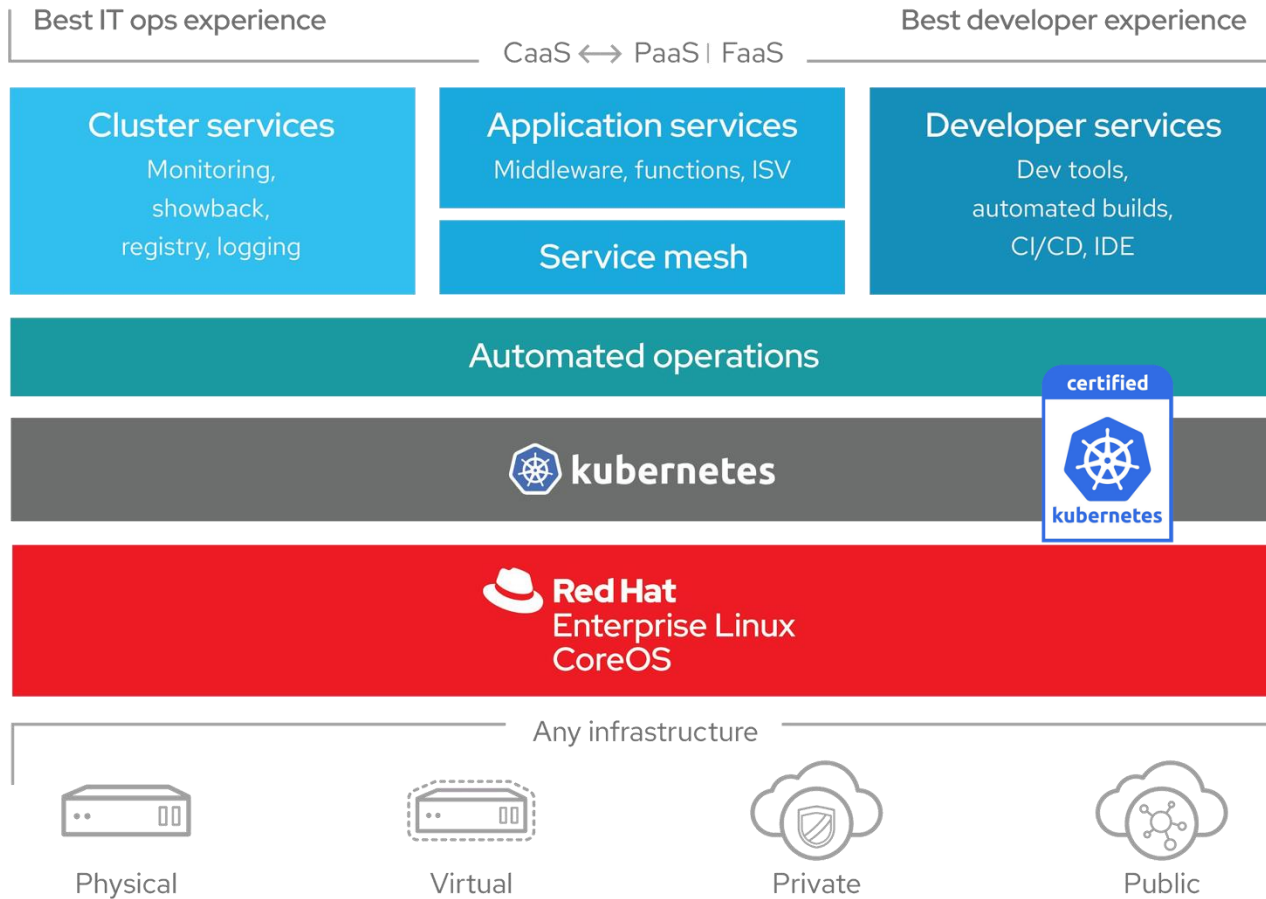


OPENSIFT IS ENTERPRISE KUBERNETES

Red Hat makes building applications with containers easy



OpenShift – A smarter Kubernetes platform



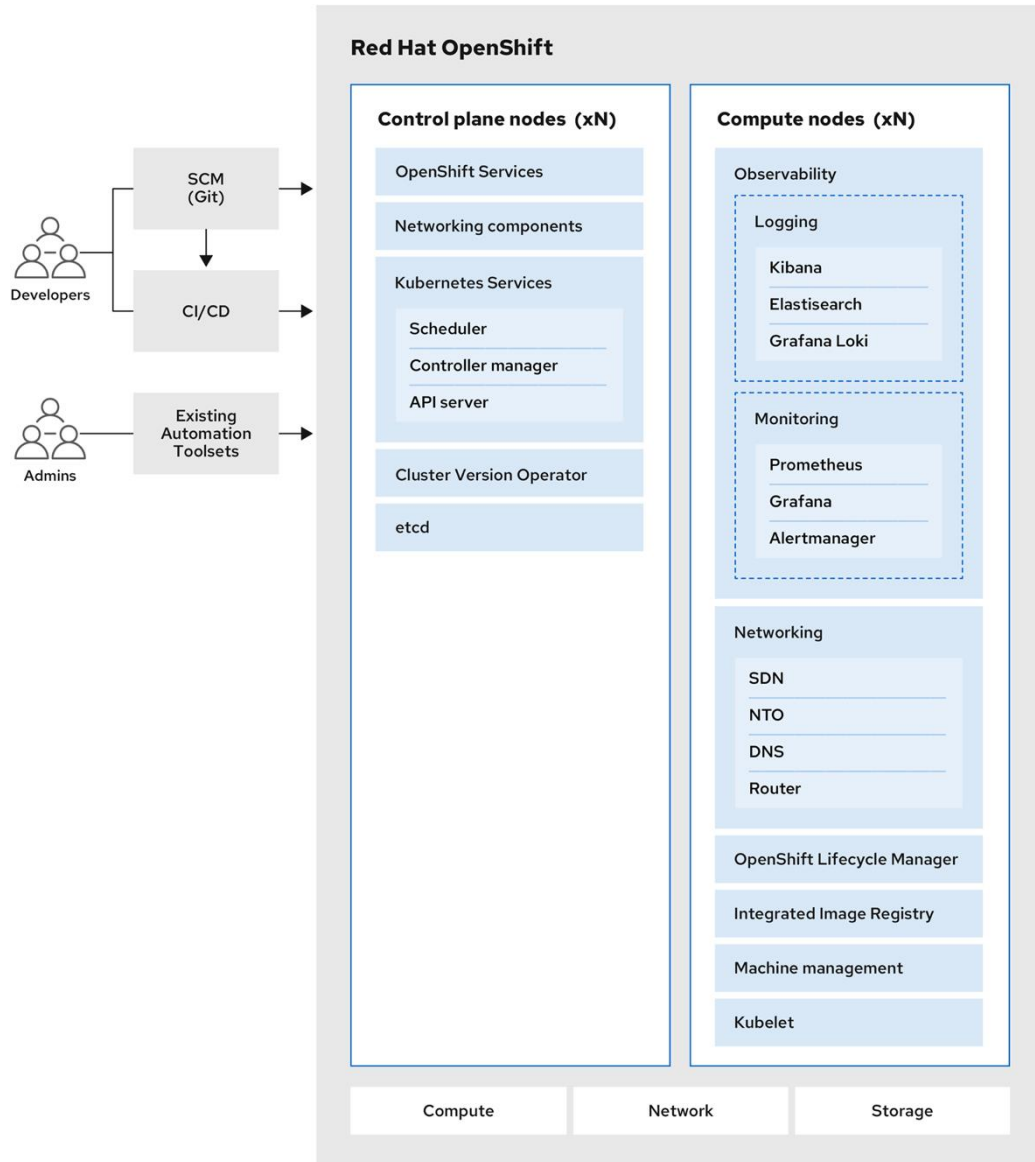
Automated, full-stack installation from the container host to application services

Seamless Kubernetes deployment to any cloud or on-premises environment

Autoscaling of cloud resources

One-click updates for platform, services, and applications

OpenShift – Architecture



596_OpenShift_0324

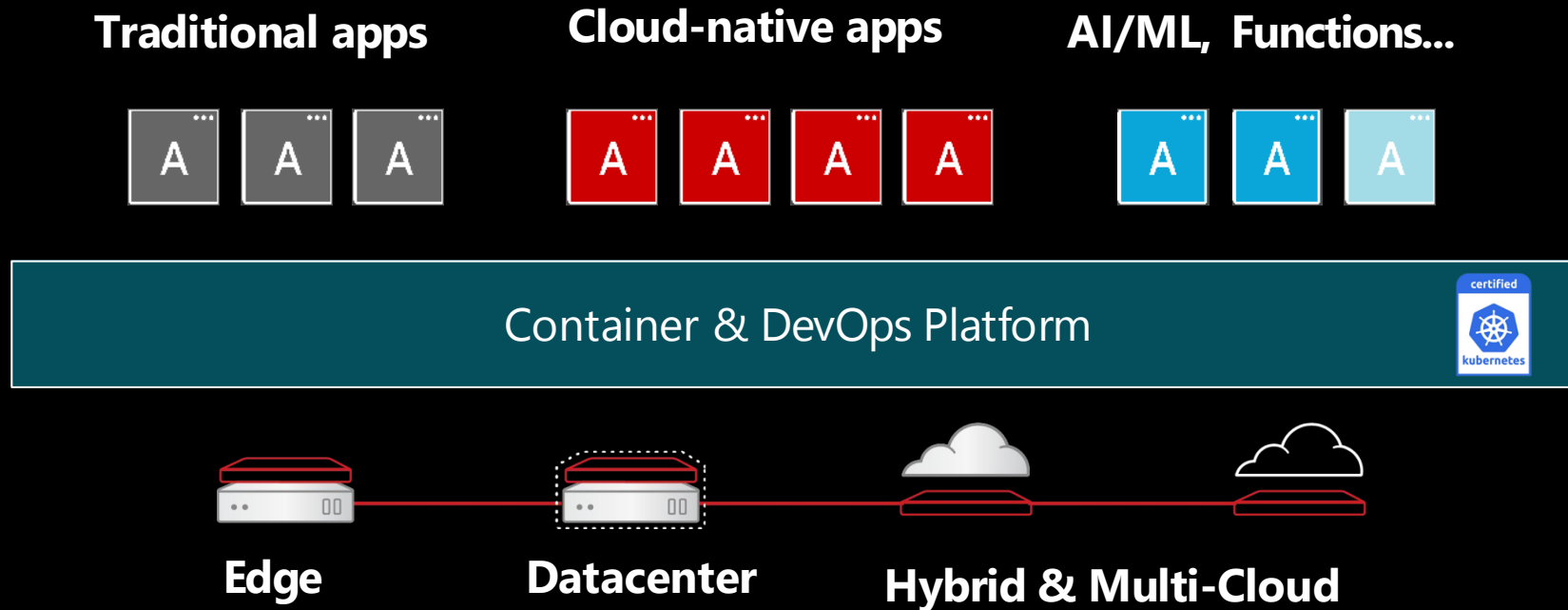
Automated, full-stack installation from the container host to application services

Seamless Kubernetes deployment to any cloud or on-premises environment

Autoscaling of cloud resources

One-click updates for platform, services, and applications

With OpenShift you can deliver all your applications in a whole new way



A consistent container application platform FROM YOUR DATACENTER TO THE CLOUD



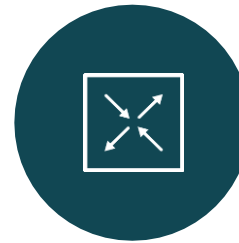
Automated
operations



Multi-tenant



Secure by
default



Network
traffic
control



Over-the-air
updates



Monitoring
& chargeback



Pluggable
architecture

Bare metal, VMware vSphere, Red Hat Virtualization, Red Hat OpenStack Platform,
Amazon Web Services, Microsoft Azure, Google

Kubernetes adoption phases

1. Stateless apps

ReplicaSets
Deployments

2. Stateful apps

StatefulSets
Storage/CSI

3. Distributed systems

Data rebalancing
Autoscaling
Seamless upgrades

Automated container operations

FULLY AUTOMATED DAY-1 AND DAY-2 OPERATIONS

INSTALL	DEPLOY	HARDEN	OPERATE
AUTOMATED OPERATIONS			
Infra provisioning	Full-stack deployment	Secure defaults	Multicluster
Embedded	On-premises and cloud	Network isolation	Monitoring and
	Unified experience	Audit and logs	Full-stack patch & upgrade
		Signing and	Zero-downtime
			Vulnerability

Self-service for developers

Home

Projects

Status

Search

Events

Catalog

Developer Catalog

Installed Operators

OperatorHub

Operator Management

Workloads

Pods

Deployments

Deployment Configs

Stateful Sets

Secrets

Config Maps

Cron Jobs

Jobs

Daemon Sets

Replica Sets

Project: production-api-backend

amqstreams.v1.1.0 > Kafka Details

production-api-kafka

Overview

YAML

Resources

0 Route

4 Service

2 StatefulSet

1 Deployment

1 ReplicaSet

7 Pods

NAME	TYPE	STATUS
production-api-kafka-clients-ca	Secret	Created
production-api-kafka-clients-ca-cert	Secret	Created
production-api-kafka-cluster-ca	Secret	Created
production-api-kafka-cluster-ca-cert	Secret	Created
production-api-kafka-cluster-operator-certs	Secret	Created
production-api-kafka-entity-operator	Deployment	Created
production-api-kafka-entity-operator-6d499d47db	ReplicaSet	Created
production-api-kafka-entity-operator-6d499d47db-82xll	Pod	Running

MongoDB Replica Set

Provided by MongoDB, Inc

Create

MongoDB Replica Set Deployment

This resource is provided by MongoDB, a Kubernetes Operator enabled by the MongoDB Operator.

Documentation

https://docs.opsmanager.mongodb.com/current/tutorial/install-k8s-

PROVIDER

MongoDB, Inc

CREATED AT

Apr 29, 2:50 pm

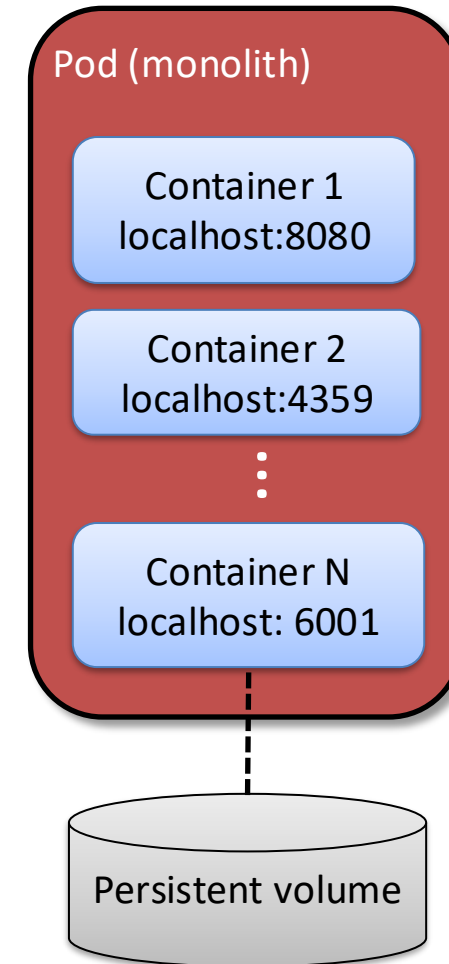
```
apiVersion:
mongodb.com/v1
kind:
MongoDbReplicaSet
metadata:
  name: example
  namespace:
production
spec:
  members: 3
  version: 4.0.2
```



Kubernetes Resources

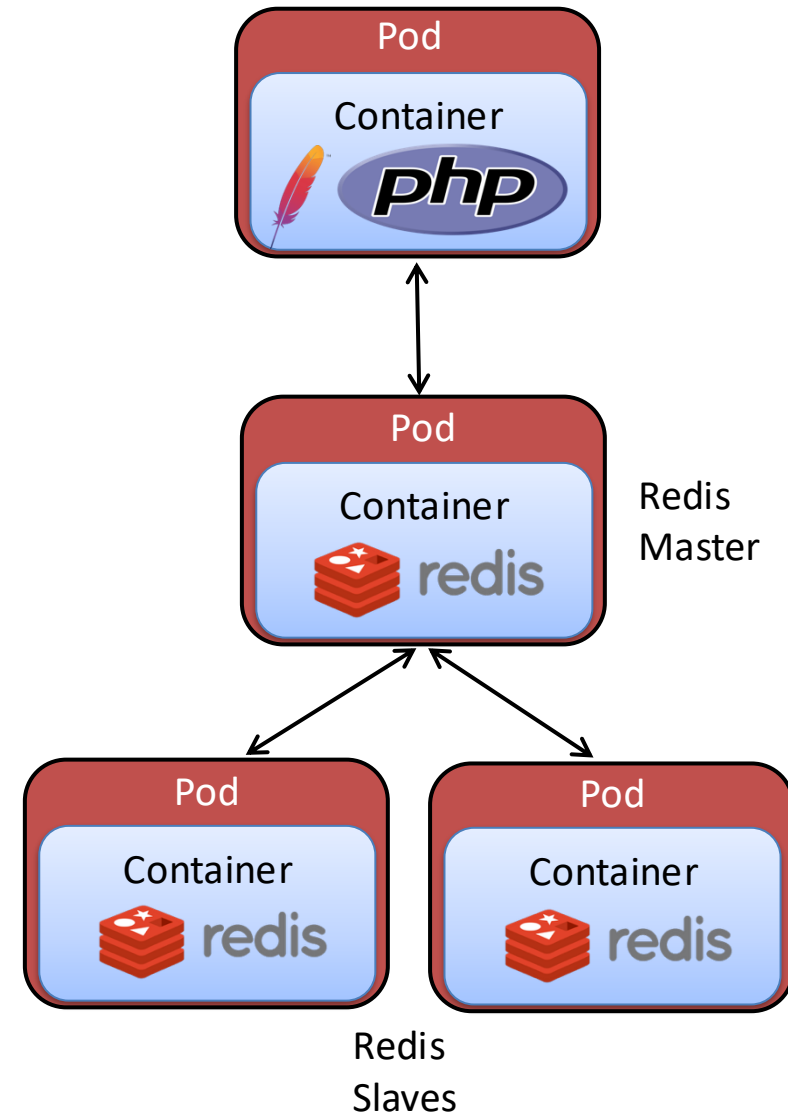
Pods Enable Deployment Flexibility

- Possible to use a single pod to run a monolithic application
 - Each application process can be built as a container
 - All containers can access each other's ports on localhost
- More advanced features of the K8s system available if application built instead from assemblages of pods, e.g.
 - Web tier: Apache pods
 - Data tier: Redis master/slave pods
- Pods provide scale and elasticity via replication – not possible in the monolith
- Best practice: assume every pod is mortal



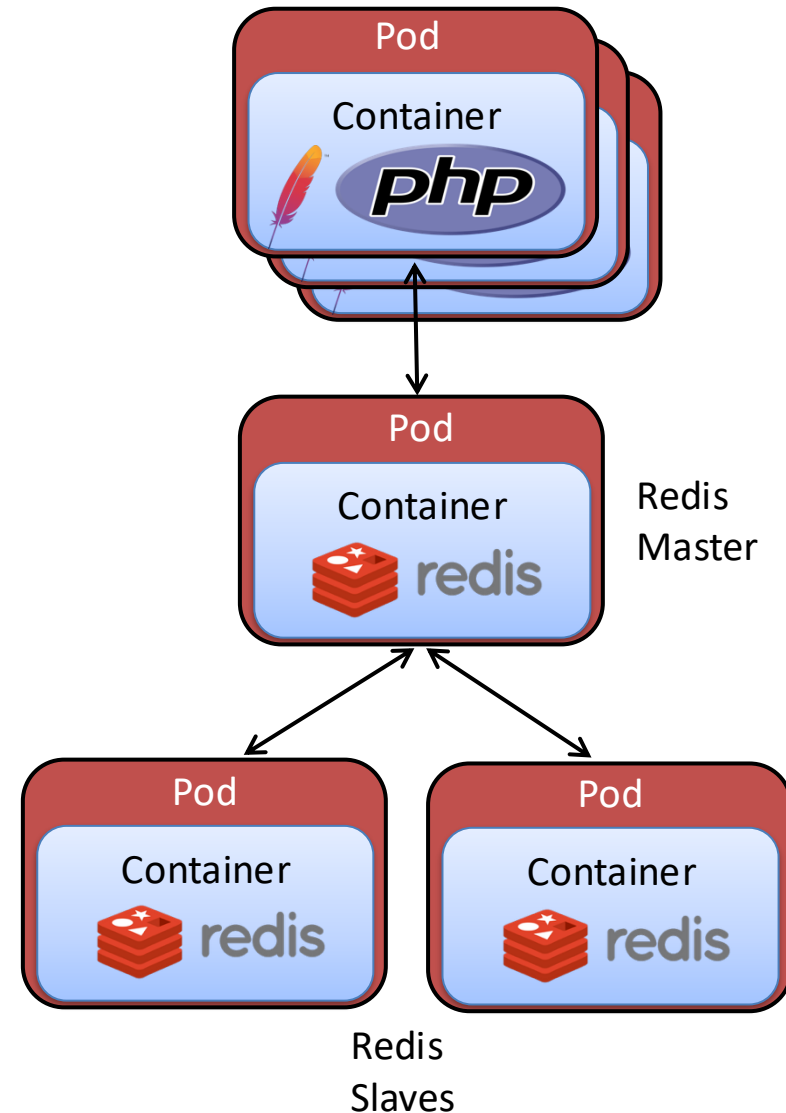
Pods Enable Deployment Flexibility

- Possible to use a single pod to run a monolithic application
 - Each application process can be built as a container
 - All containers can access each other's ports on localhost
- More advanced features of the K8s system available if application built instead from assemblages of pods, e.g.
 - Web tier: Apache pods
 - Data tier: Redis master/slave pods
- Pods provide scale and elasticity via replication – not possible in the monolith
- Best practice: assume every pod is mortal



Pods Enable Deployment Flexibility

- Possible to use a single pod to run a monolithic application
 - Each application process can be built as a container
 - All containers can access each other's ports on localhost
- More advanced features of the K8s system available if application built instead from assemblages of pods, e.g.
 - Web tier: Apache pods
 - Data tier: Redis master/slave pods
- Pods provide scale and elasticity via replication – not possible in the monolith
- Best practice: assume every pod is mortal



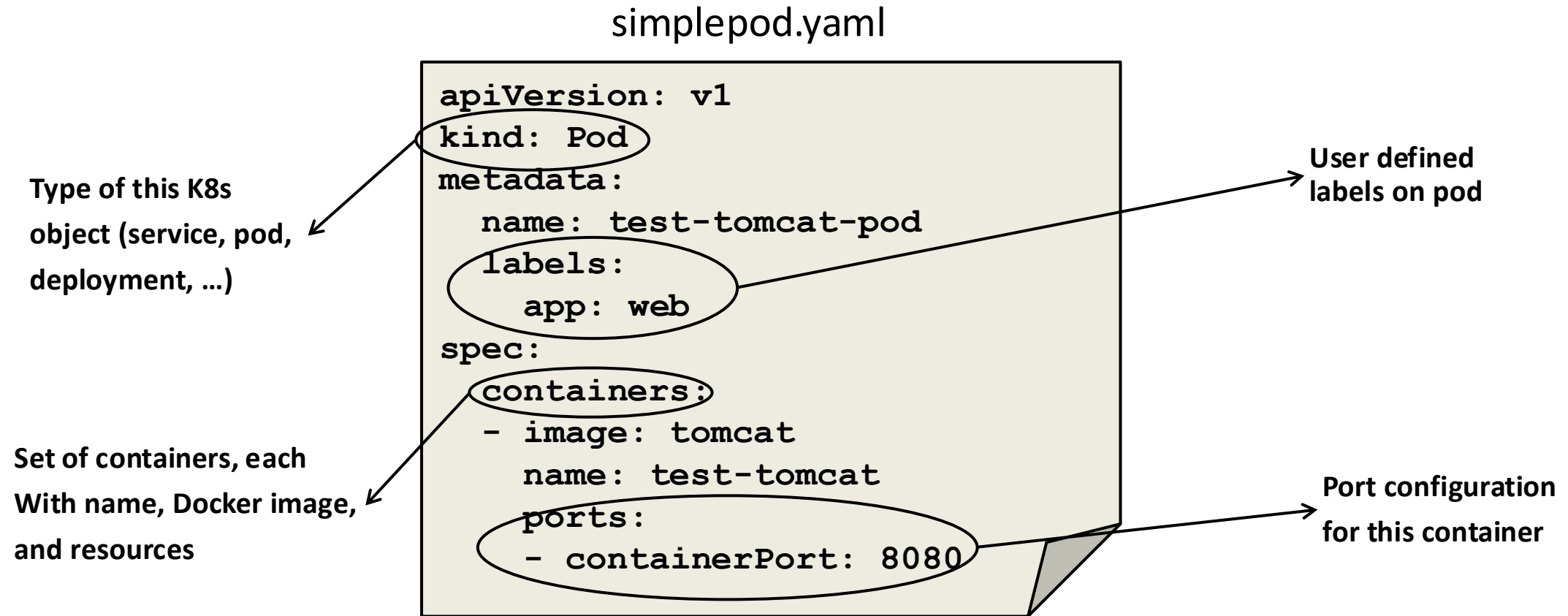
Defining a Pod via a Manifest File

Like other K8s objects, pods can be defined in YAML or JSON files

- K8s API accepts object definitions in JSON, but manifests often in YAML
- YAML format used by a variety of other tools, e.g. Docker Compose, Ansible, etc.
- **kind** field value is 'Pod'
- **metadata** includes
 - **name** to assign to pod
 - **label** values
- **spec** includes specifics of container images, ports, and other resources

```
apiVersion: v1
kind: Pod
metadata:
  name: test-tomcat-pod
  labels:
    app: web
spec:
  containers:
  - image: tomcat
    name: test-tomcat
    ports:
    - containerPort: 8080
```

Looking at a Pod Manifest File



- Configuration options similar to creating Docker container directly

Defining a Pod with Multiple Containers

```
apiVersion: v1
kind: Pod
metadata:
  name: test-tomcat-pod
spec:
  containers:
  - image: tomcat
    name: test-tomcat
    ports:
    - containerPort: 8080
  - image: mysql
    name: test-mysql
    ports:
    - containerPort: 3306
```

→ multipod.yaml

- Pod spec can contain multiple containers from different images
- Containers in pod share local network context and cluster IP for pod

Deployments



What is a Deployment?

Kubernetes controller optimal for stateless applications

- Deployments allow you to declaratively manage pods, including replication
- Deployments support
 - Creating, rolling out, and rolling back changes to homogeneous set of pods
 - Scaling set of pods out and back declaratively
- Deployments include
 - Implicit Replica Set controller to handle pod replicas
 - Template spec of pods to be created and managed – no need to separately create pods
- Deployments used for web applications, mobile back-ends, API's

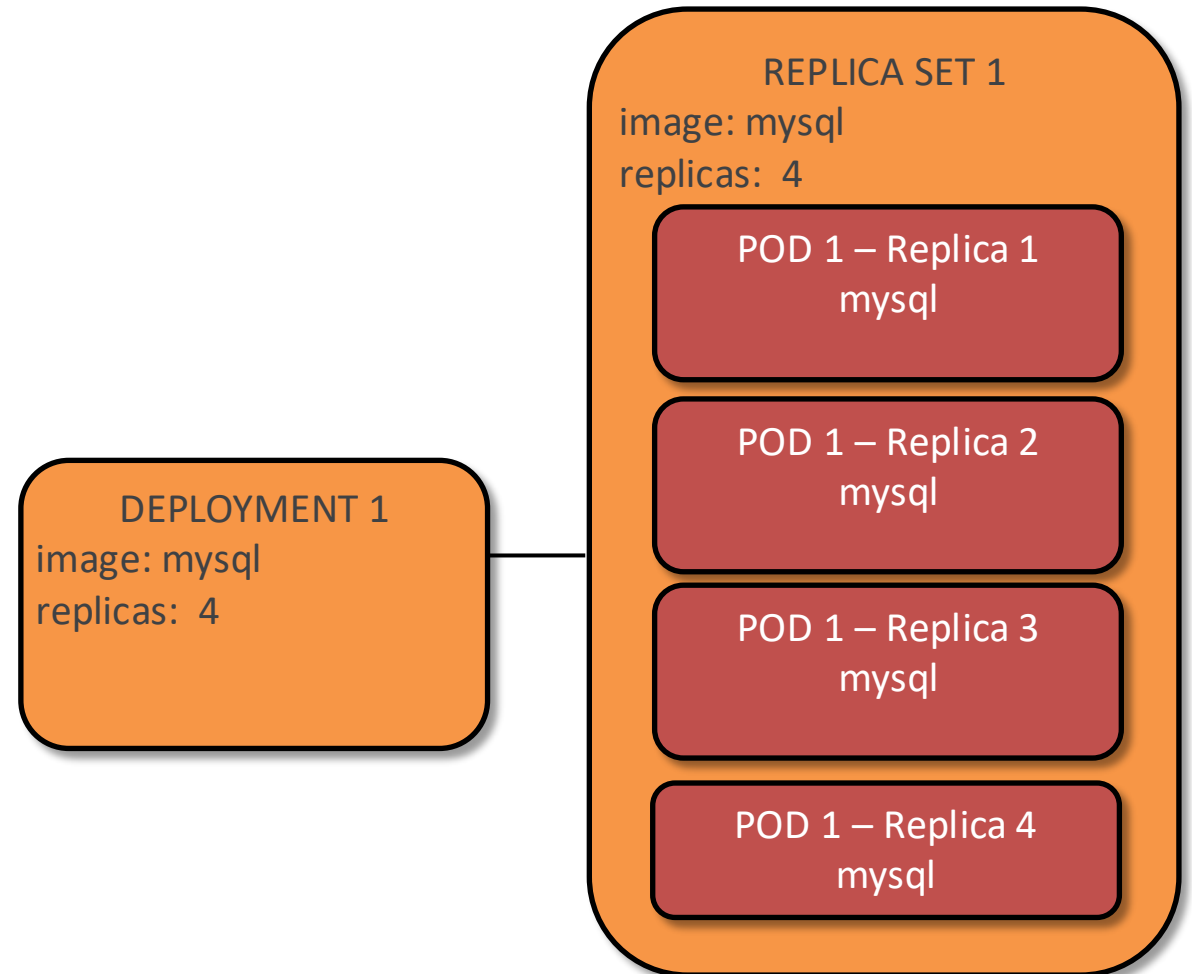
What if my Application isn't Stateless?

- Kubernetes provides other controller objects for applications that need different deployment schemes
- **StatefulSets** (previously PetSets) control deployment of pods for applications that need more stable deployment contexts
 - Pods in StatefulSets have unique ordinal, stable network identity and stable storage using persistent volumes
 - When pods are deployed, they are created in sequence of ordinals 1..N
 - Pod N must be running and ready before Pod N+1 is deployed
 - When pods are destroyed, they are terminated in reverse sequence N..1
- **DaemonSets** ensure that a replica of a specified pod is running on every node (or every selected node) in the cluster
- **Jobs** manage sets of pods where N must run to successful completion

Deployments Control ReplicaSet Controllers

Definition of how many replicated Pods should exist

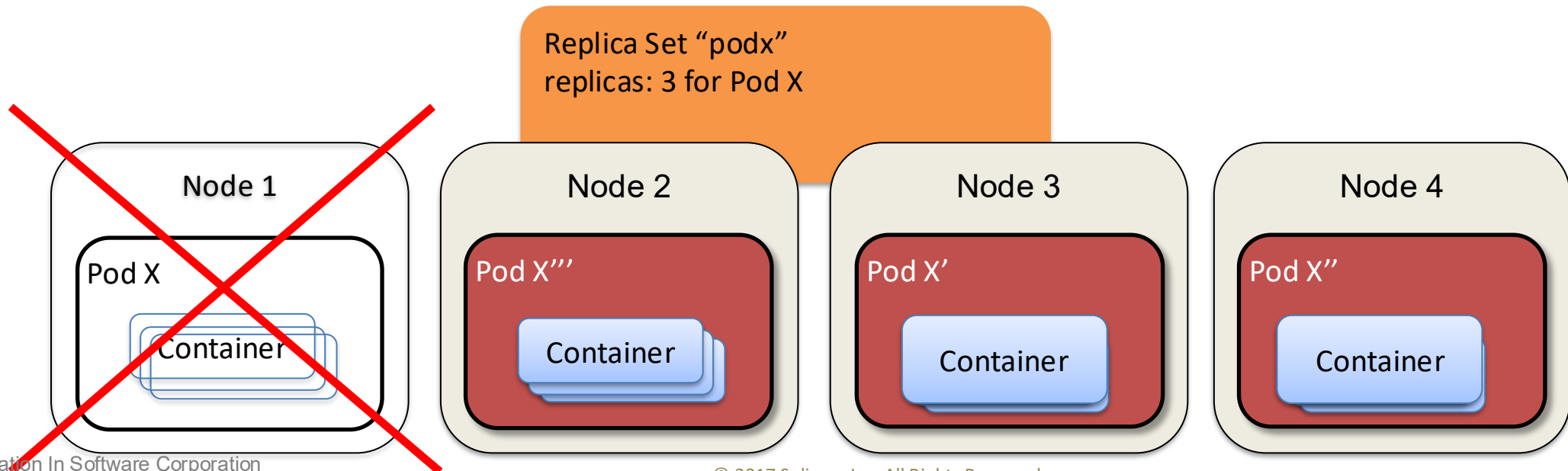
- Deployment creates and manages a Replica Set that manages a set of pods
- Replica count can be adjusted as needed to scale the Replica Set out and back
- Replica Set successor to the ReplicationController object



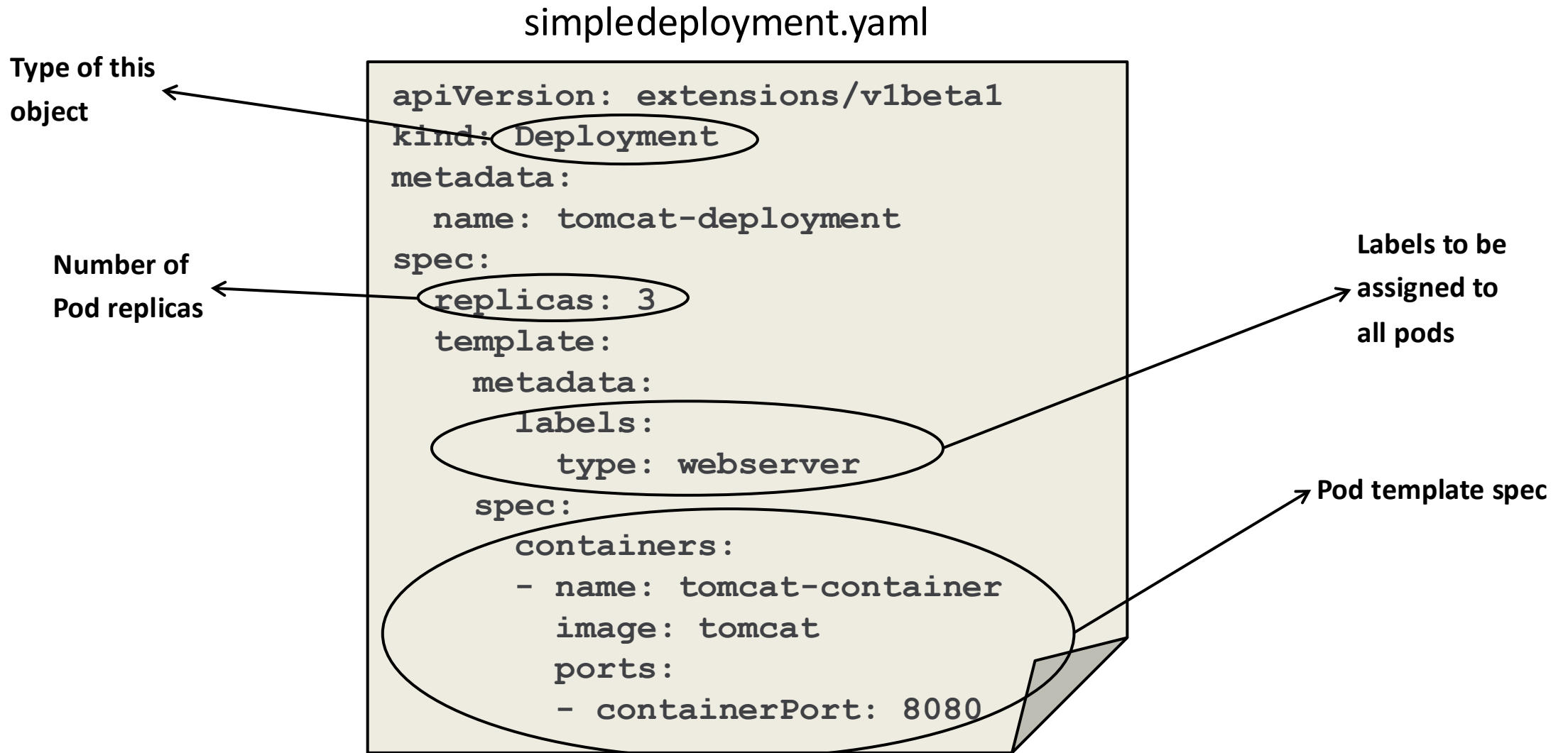
What is a Replica Set?

Provides scaling and high availability

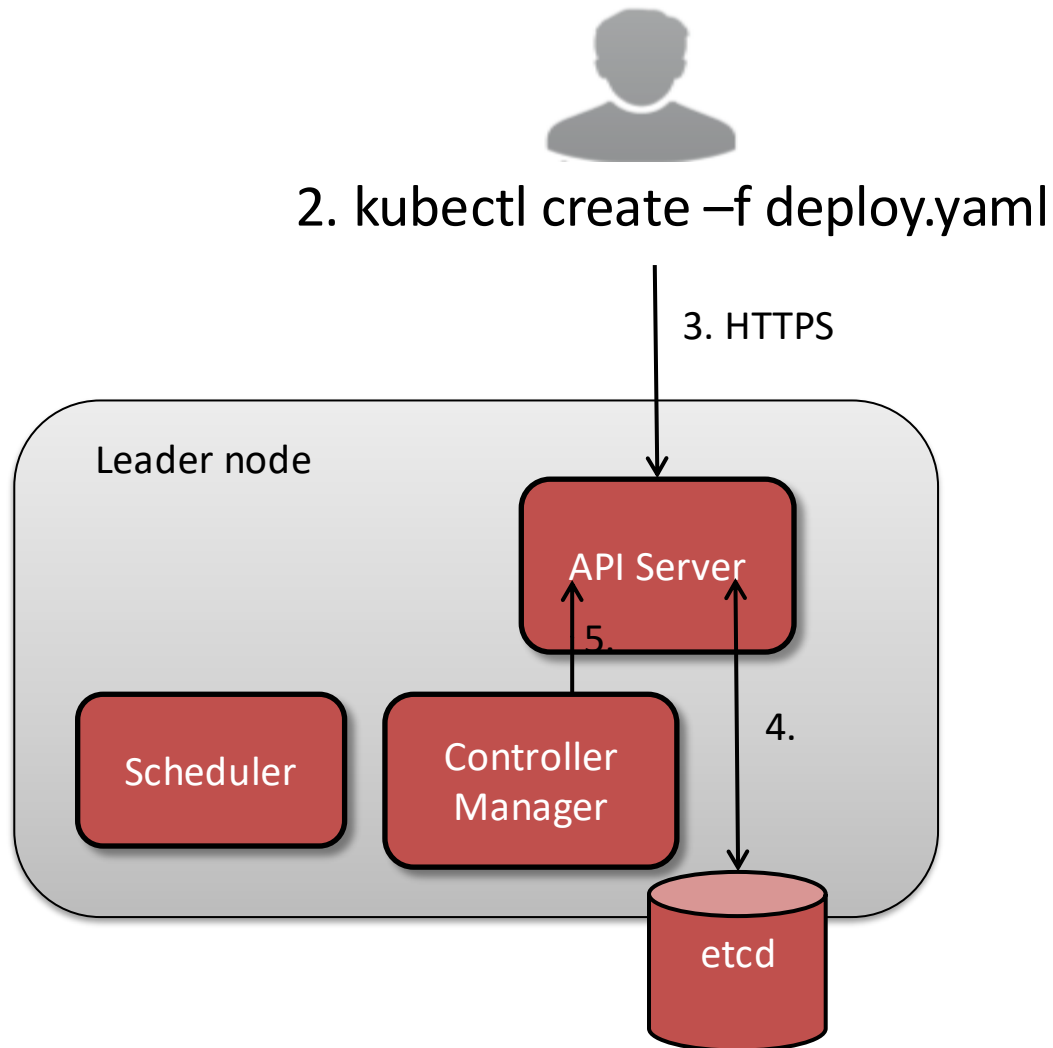
- Replica count can be changed to provide scaling on demand as needed
- If the node hosting a pod fails, the Kubernetes cluster will recreate the pod elsewhere to achieve the target number of replicas



Examining a Deployment Manifest File

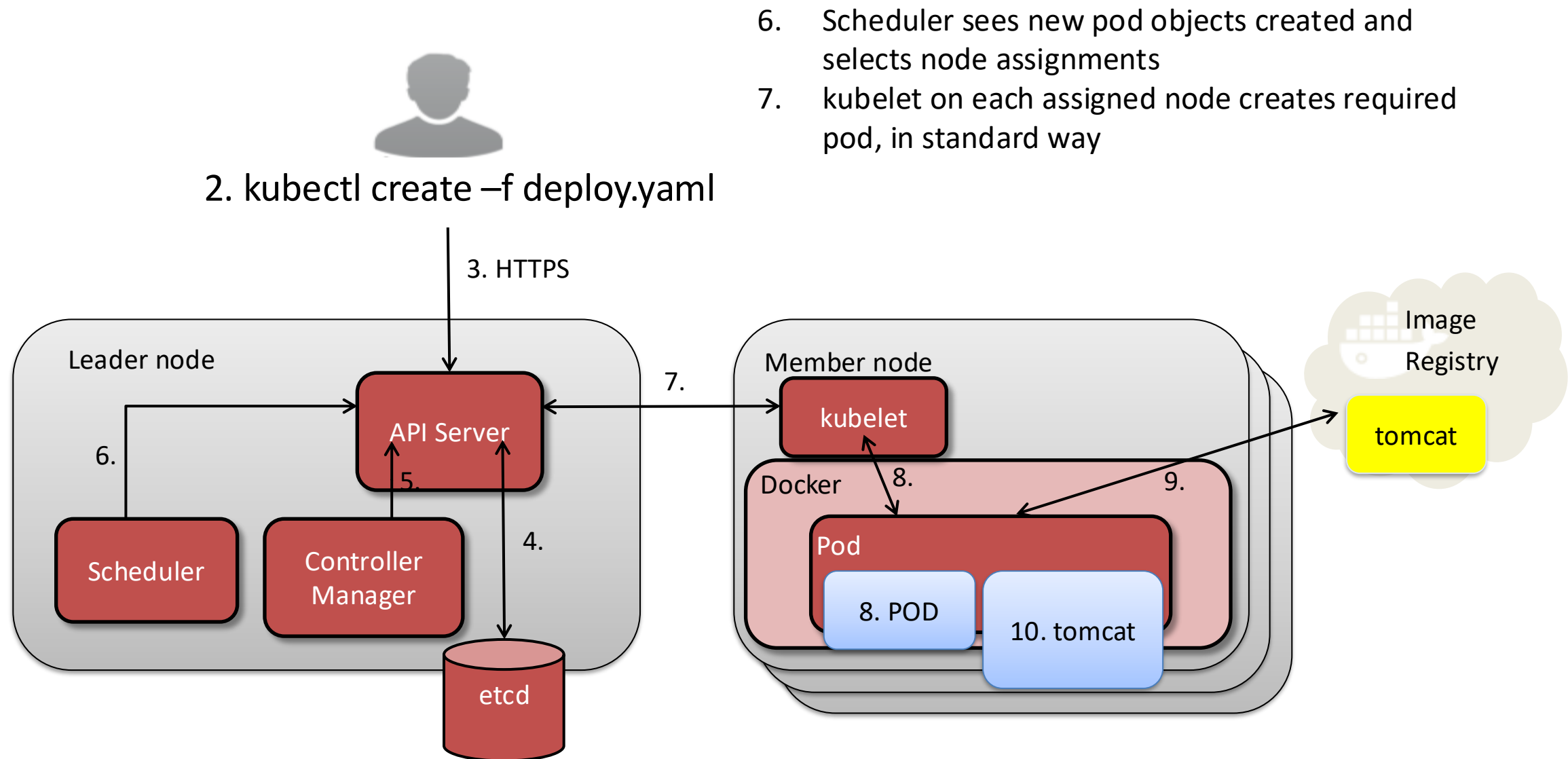


Deployment Creation Process



1. User writes a deployment manifest file
2. User requests creation of deployment from manifest via CLI
3. CLI tool marshals parameters into K8s RESTful API request (HTTP POST)
4. kube-apiserver creates new deployment object record in etcd, and new Replica Set object
5. kube-controller-manager sees new Replica Set and
 - a. Evaluates state of existing vs. required replicas
 - b. Submits pod creation requests to API to create required number of replicas

Deployment Creation Process



Kubernetes Services

Kubernetes Service Objects:Microservices

Services provide abstraction between layers of an application

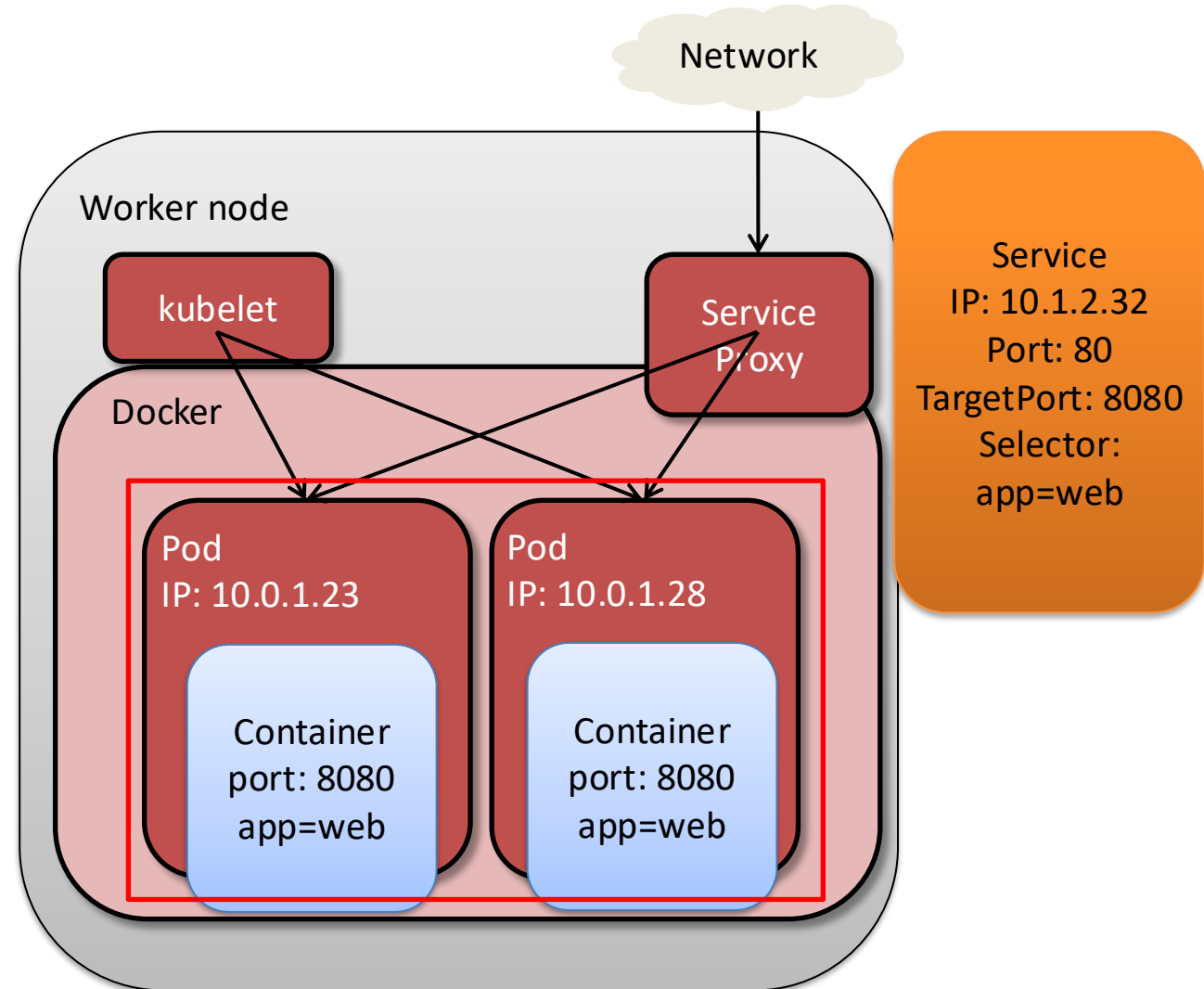
- **Service** object provides a stable IP for a collection of Pods
- Services use a label **selector** to target a specific set of pods as endpoints to receive proxied traffic
- Clients can reliably connect via the service IP and port(s), even as individual endpoint pods are dynamically created & destroyed
- Can model other types of backends using services without selectors

```
apiVersion: v1
kind: Service
metadata:
  name: wordpress
  labels:
    app: wordpress
spec:
  ports:
    - port: 80
      targetPort: 8080
  selector:
    app: wordpress
    tier: frontend
  type: LoadBalancer
```

sampleservice.yaml

Services Provide Abstraction Layer for Applications

- Services can be used for communications between application tiers
- Services can also be used to expose applications outside the K8s cluster
- Services distribute requests over the set of Pods matching the service's selector
 - Service functions as TCP and UDP proxy for traffic to Pods
 - Service maps its defined ports to listening ports on Pod endpoints



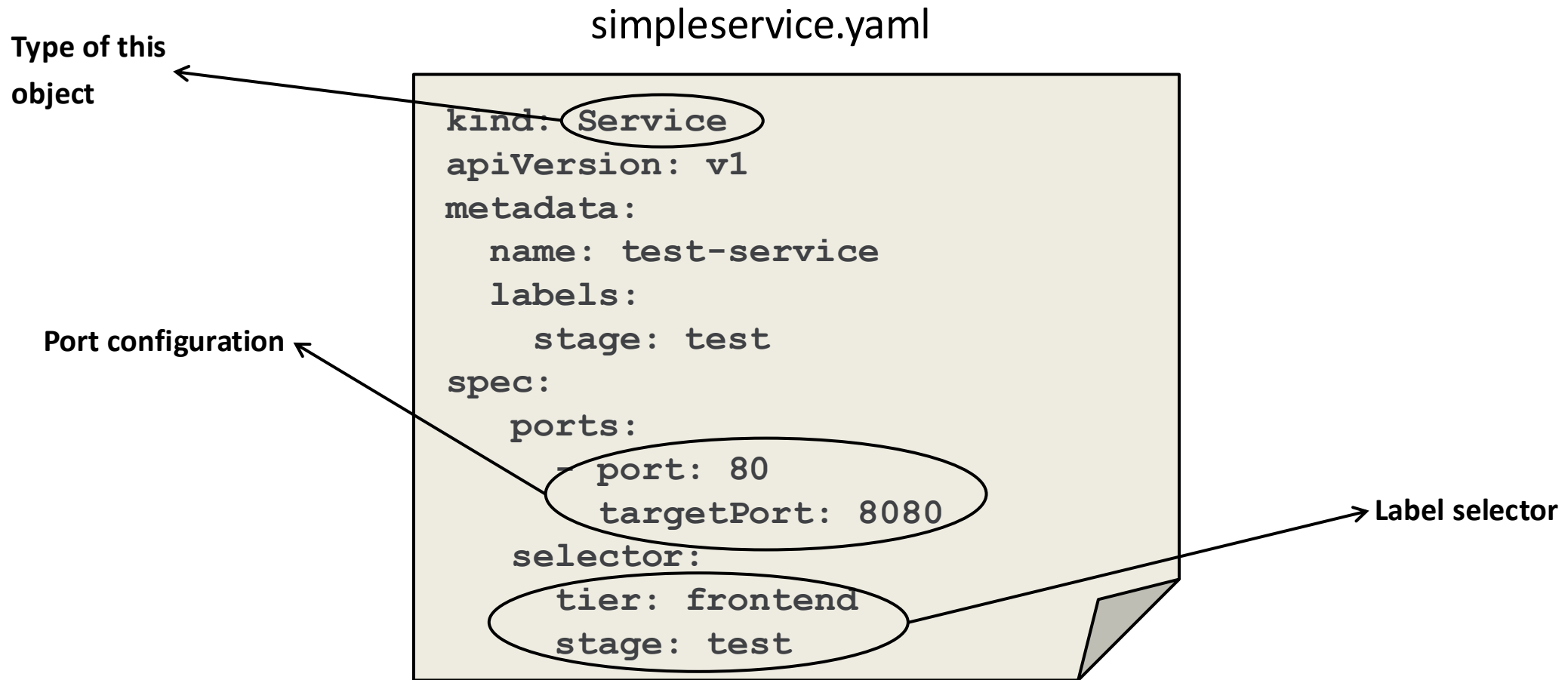
Defining a Service Using a Manifest File

Services can be defined in YAML or JSON, like other K8s resources

- **kind** field value is 'Service'
- **metadata** includes
 - **name** to assign to Service
- **spec** includes the ports associated with the Service
 - **port** is the Service's port value
 - **targetPort** is connection port on selected pods (default: **port** value)
- **selector** specifies a set of label KV pairs to identify the endpoint pods for the Service

```
kind: Service
apiVersion: v1
metadata:
  name: test-service
  labels:
    stage: test
spec:
  ports:
    - port: 80
      targetPort: 8080
  selector:
    tier: frontend
    stage: test
```

Reviewing a Service Manifest File



ServiceTypes and Exposing Applications

- By default, a Service is assigned a cluster-internal IP – good for back-ends
 - **ServiceType** ClusterIP
- To make a front-end Service accessible outside the cluster, there are other ServiceTypes available
 - **ServiceType** NodePort exposes Service on each Node's IP on a static port
 - **ServiceType** LoadBalancer exposes the Service externally using cloud provider's load balancer
- Can also use a Service to expose an external resource via **ServiceType** ExternalName

```
kind: Service
apiVersion: v1
metadata:
  name: test-service
  labels:
    stage: test
spec:
  ports:
    - port: 80
      targetPort: 8080
      nodePort: 30080
  selector:
    tier: frontend
  type: NodePort
```

Exposing Services at L7 through Ingresses

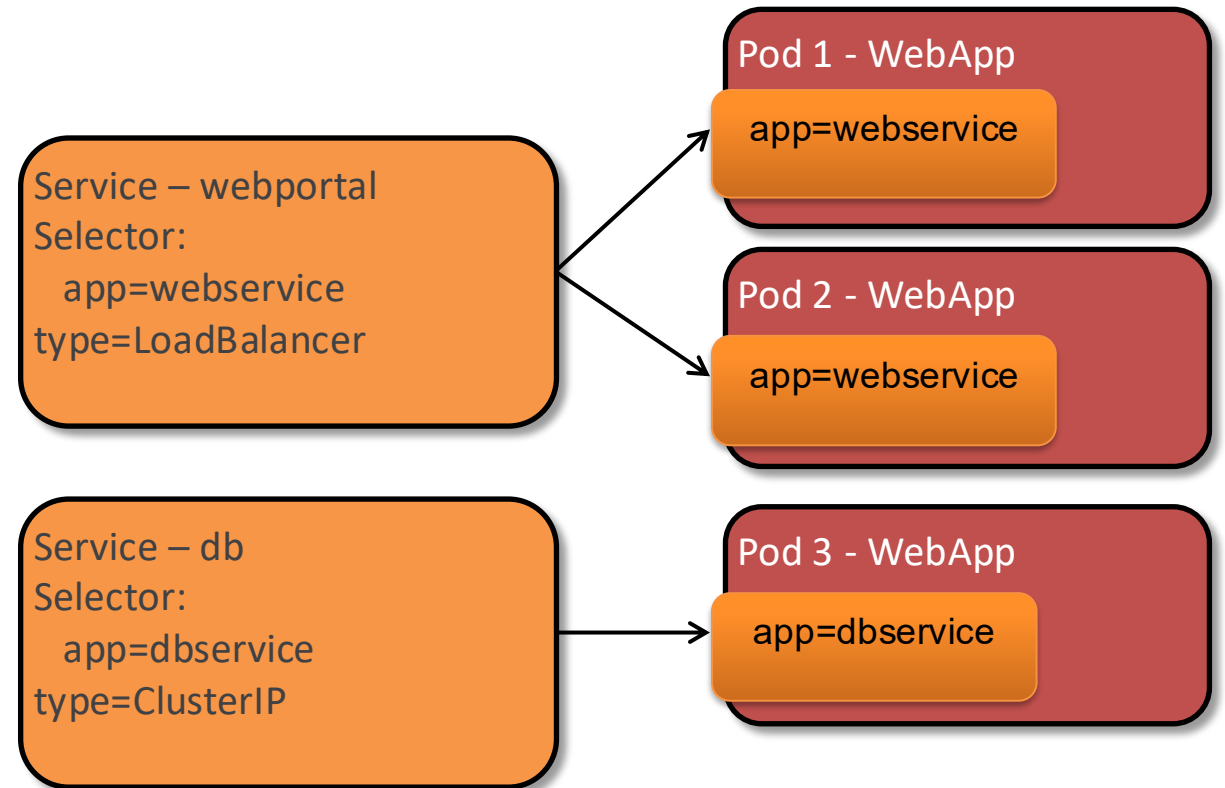
- Kubernetes also provides the facility to define **Ingress** resource to configure an external loadbalancer at L7
- Spec of Ingress resource is a set of rules matching HTTP host/url paths to specific Service backends
- Ingresses require the cluster to be running an appropriately configured Ingress controller to function (e.g. nginx)
- Useful for implementing fanout, Service backends for virtual hosts, etc.

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: test-ingress
spec:
  rules:
  - host: bar.foo.com
    http:
      paths:
      - path: /first
        backend:
          serviceName:
firstservice
          servicePort: 80
      - path: /second
        backend:
          serviceName:
secondservice
          servicePort: 80
```

Selecting Pods as Service Endpoints

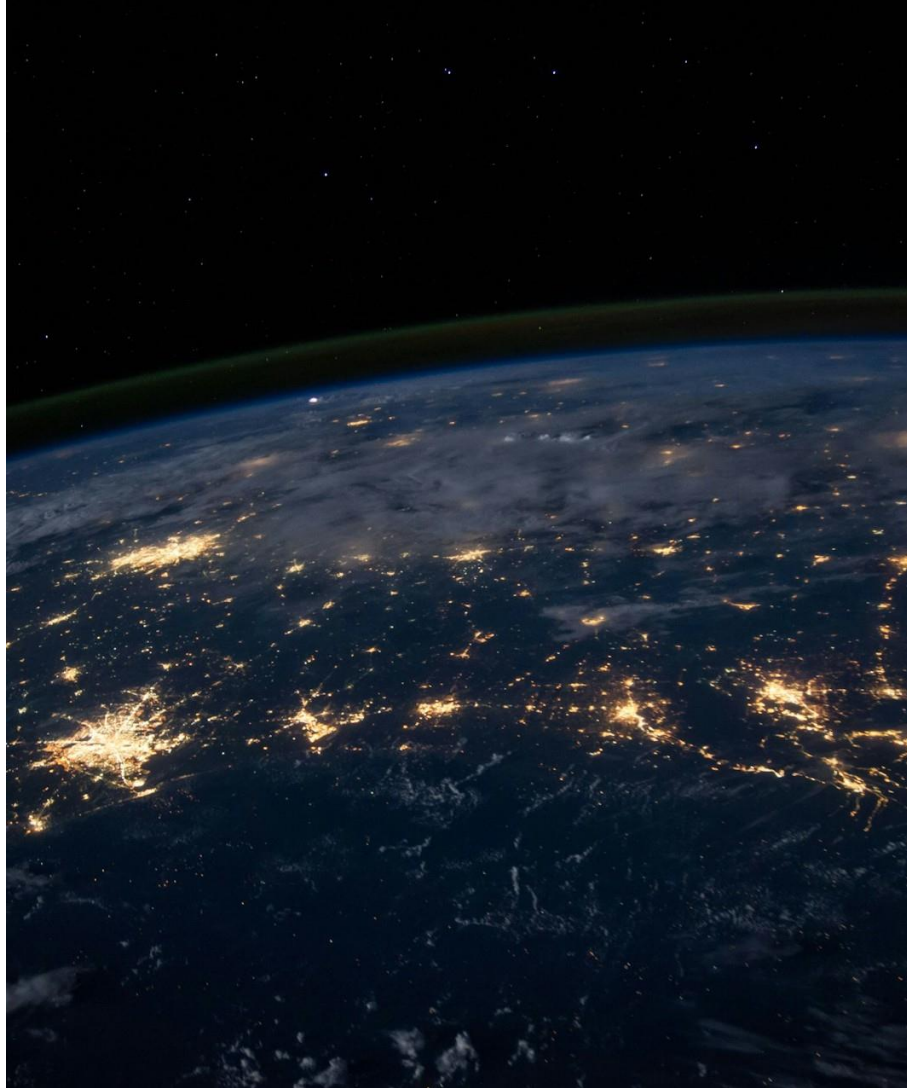
Service's pod selector based on labels

- Multiple pods can have the same label, unlike pod names which are unique in the namespace
- K8s system re-evaluates Service's selector continuously
- K8s maintains **endpoints** object of same name with list of pod IP:port's matching Service's selector



Microservices Architecture

Module Overview



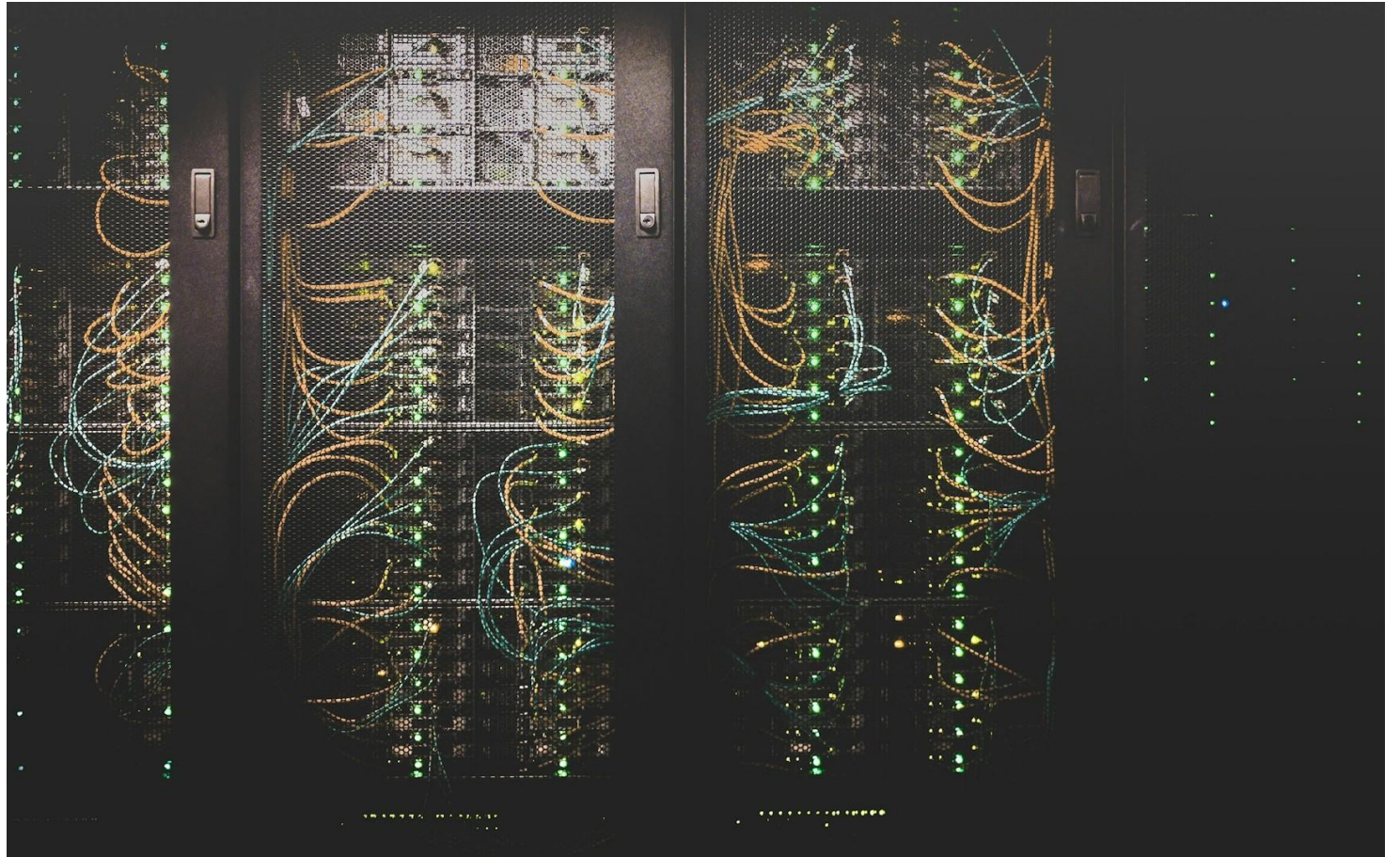
Microservices split a monolithic application into small, independently deployable services. The approach trades single-codebase simplicity for operational flexibility, but only if you get the fundamentals right.

- Stateless vs stateful design and why it determines your scaling ceiling
- Service discovery: how services find each other in dynamic environments
- Architecture patterns proven at Netflix, Uber, and Spotify
- Deployment strategies that reduce blast radius when things go wrong

Stateless vs Stateful Services

Stateless services need no memory of prior calls. Stateful services retain data between requests. This shapes your entire scaling and recovery strategy.

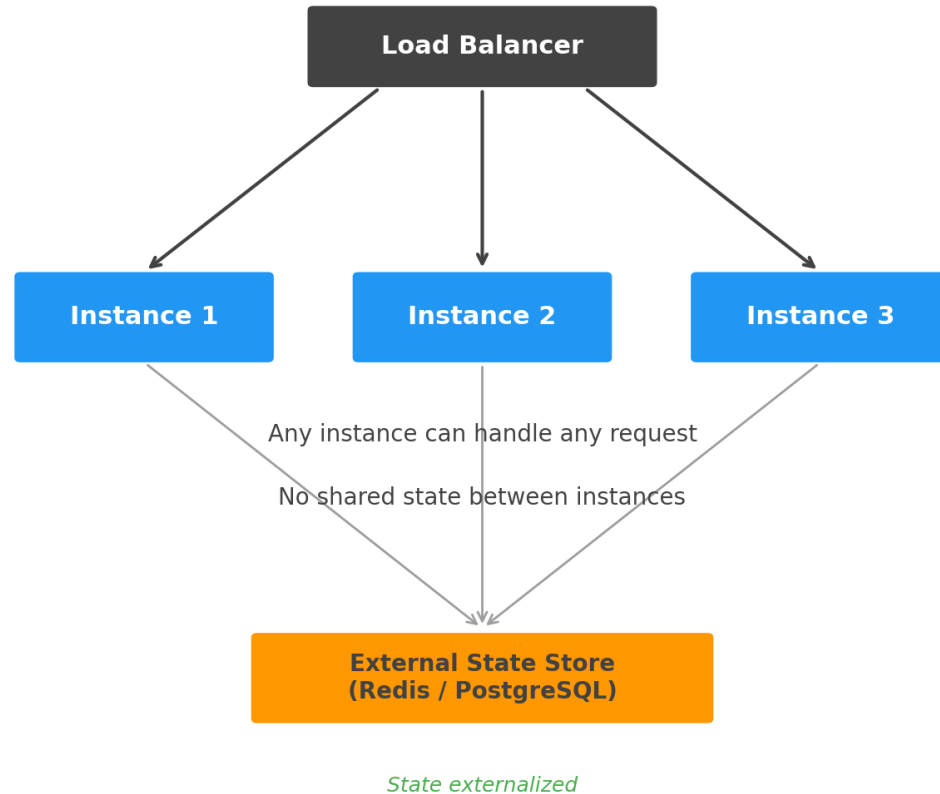
- Stateless scales by adding pods behind a load balancer. Uber runs thousands of stateless pricing pods
- Stateful needs sticky sessions or consensus protocols. Kafka and etcd in K8s remain painful
- Hybrid: stateless compute with externalized state in Redis or PostgreSQL. Most prod systems do this
- Common mistake: hiding state in local memory. Works until a pod restarts and sessions vanish



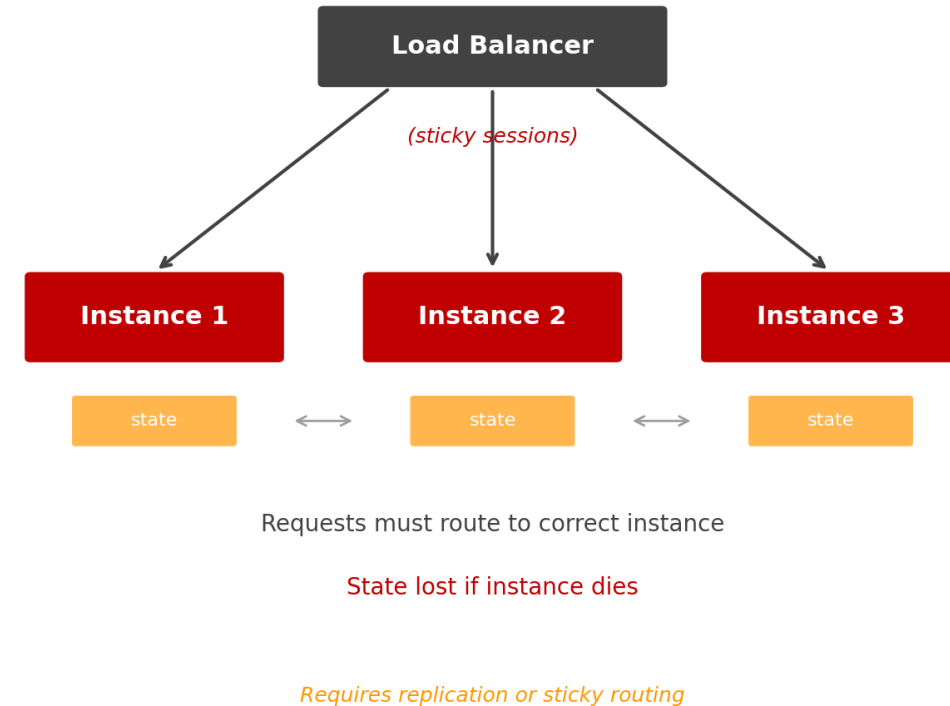
Stateless vs Stateful Architecture

Stateless services externalize state to dedicated stores. Stateful services hold state locally, requiring sticky routing and replication.

Stateless Service



Stateful Service



State Management Strategies



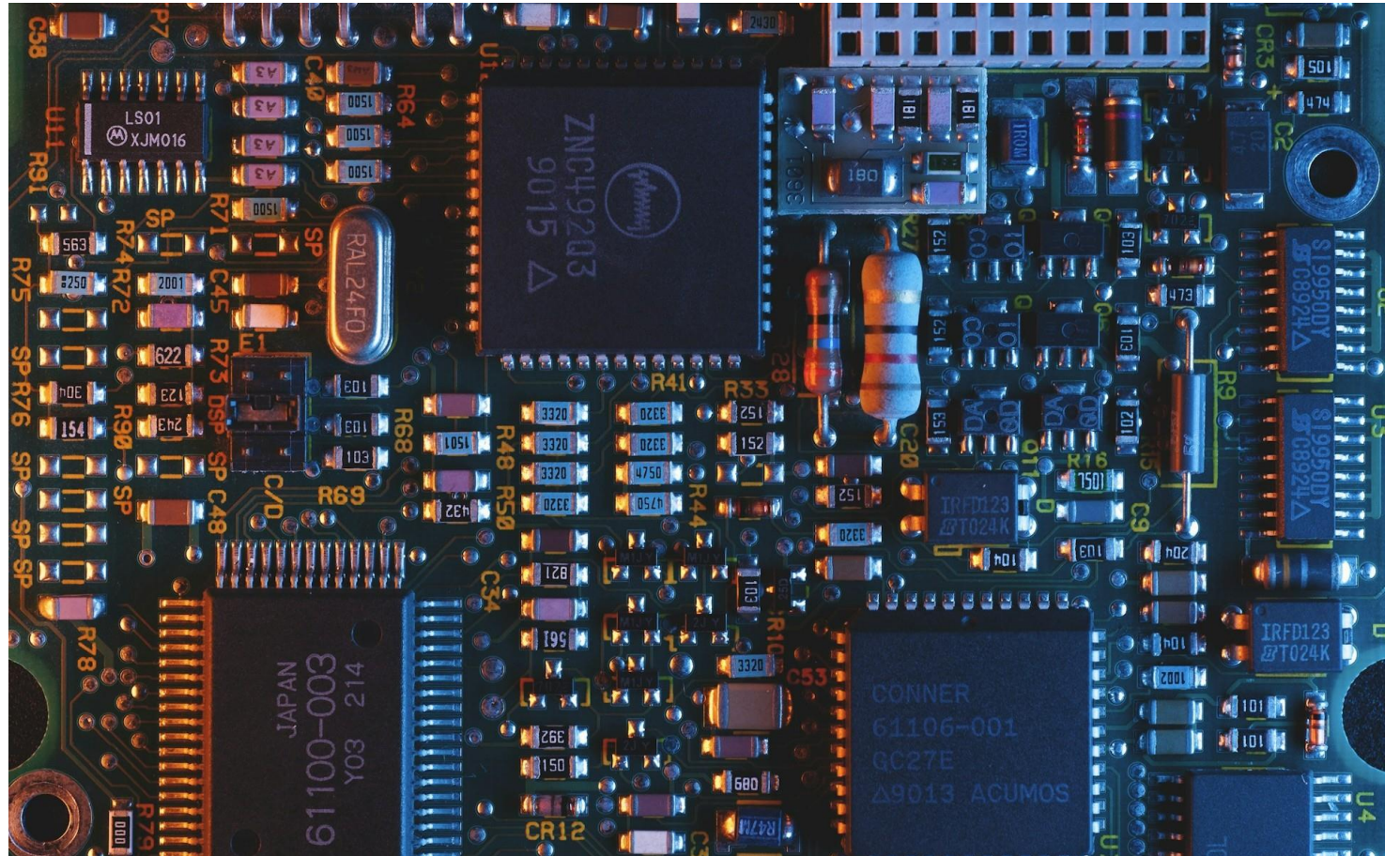
Externalized state needs a home. Each option has different consistency, latency, and failure trade-offs. Choose wrong and you create bottlenecks that are expensive to fix.

- Redis/Memcached for session data. Sub-ms reads but volatile. Shopify runs Redis Cluster at 400K+ req/sec
- Relational DBs for transactional state. Strong consistency but hard to shard. Stripe runs payments on PostgreSQL
- Event sourcing captures every change as an immutable event. Great for audit trails, costly replay at scale
- Saga pattern for cross-service transactions. Avoids 2PC but needs compensating actions for every failure step

Service Discovery Fundamentals

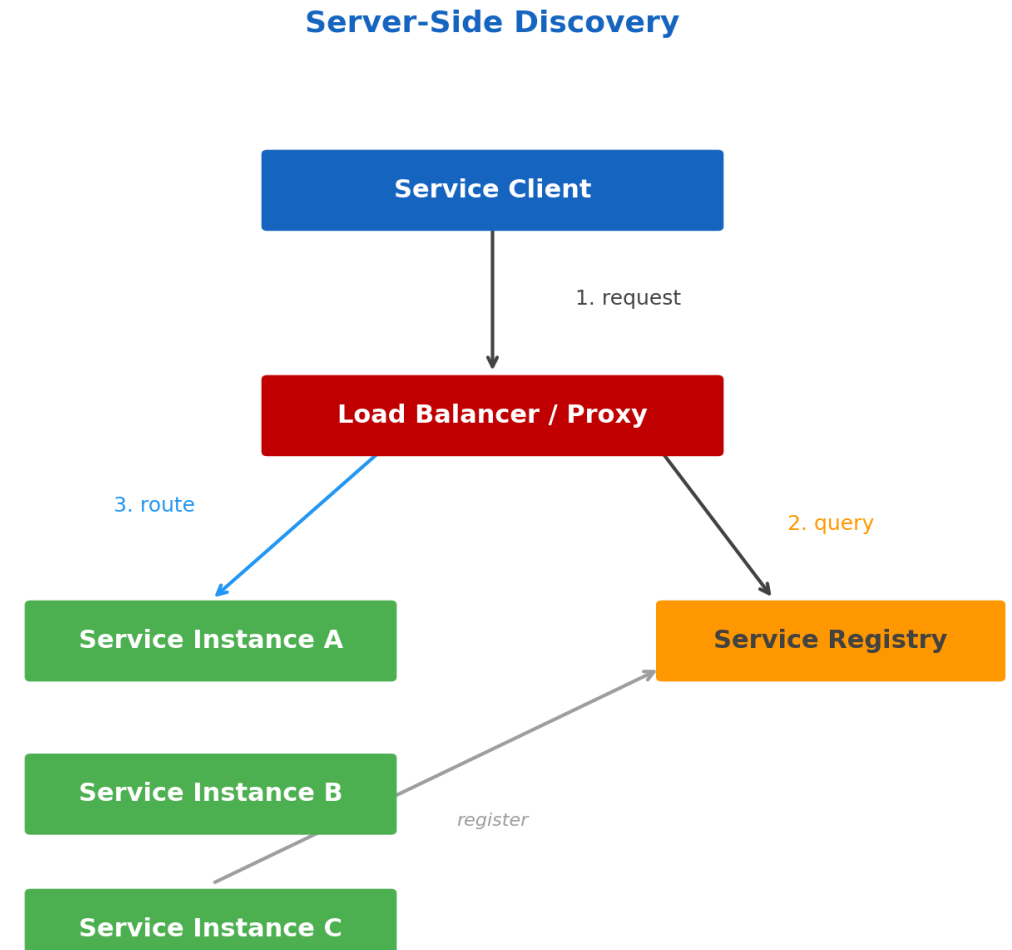
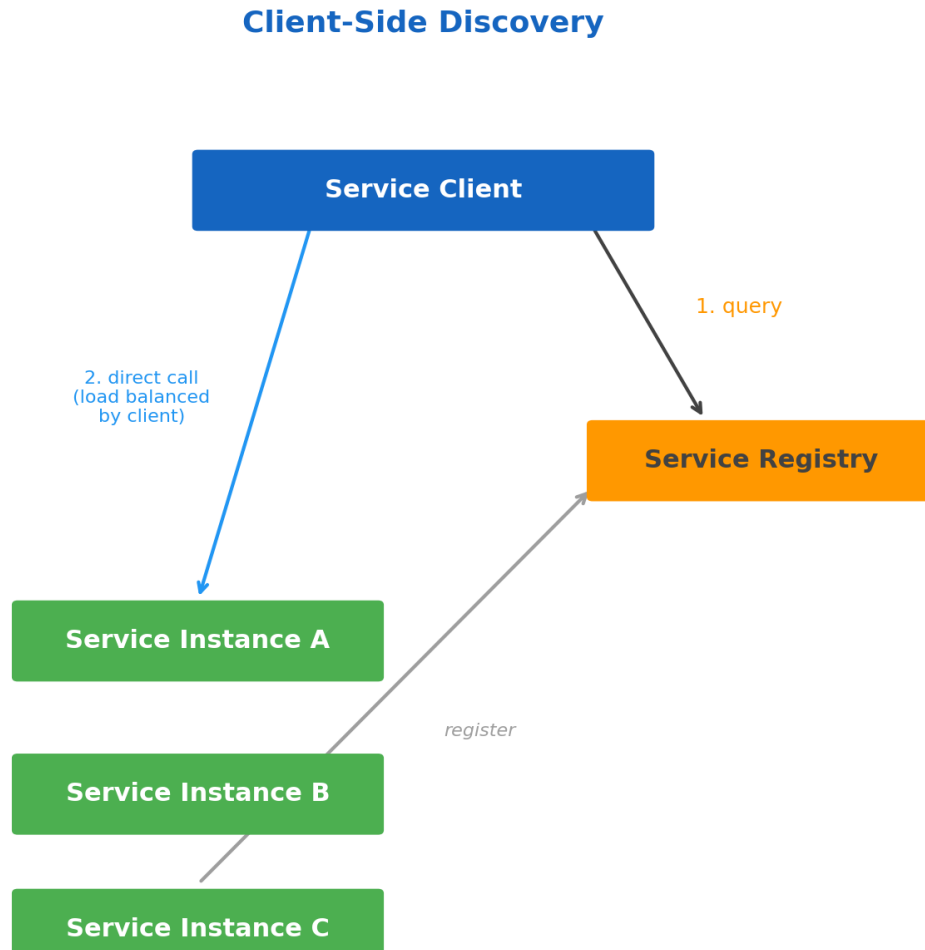
Pods start, stop, and move constantly. Hardcoded IPs break immediately. Service discovery maps names to live endpoints automatically.

- K8s Services + CoreDNS: 'payment-api' in namespace 'prod' resolves to payment-api.prod.svc.cluster.local
- Probes keep the registry honest. A pod that passes liveness but fails readiness runs but gets no traffic
- DNS TTL caching causes stale routing. After scale-down, traffic may hit dead pods for 30 seconds
- Service meshes (Istio, Linkerd) bypass DNS with sidecar proxies that get real-time endpoint updates



Client-Side vs Server-Side Discovery

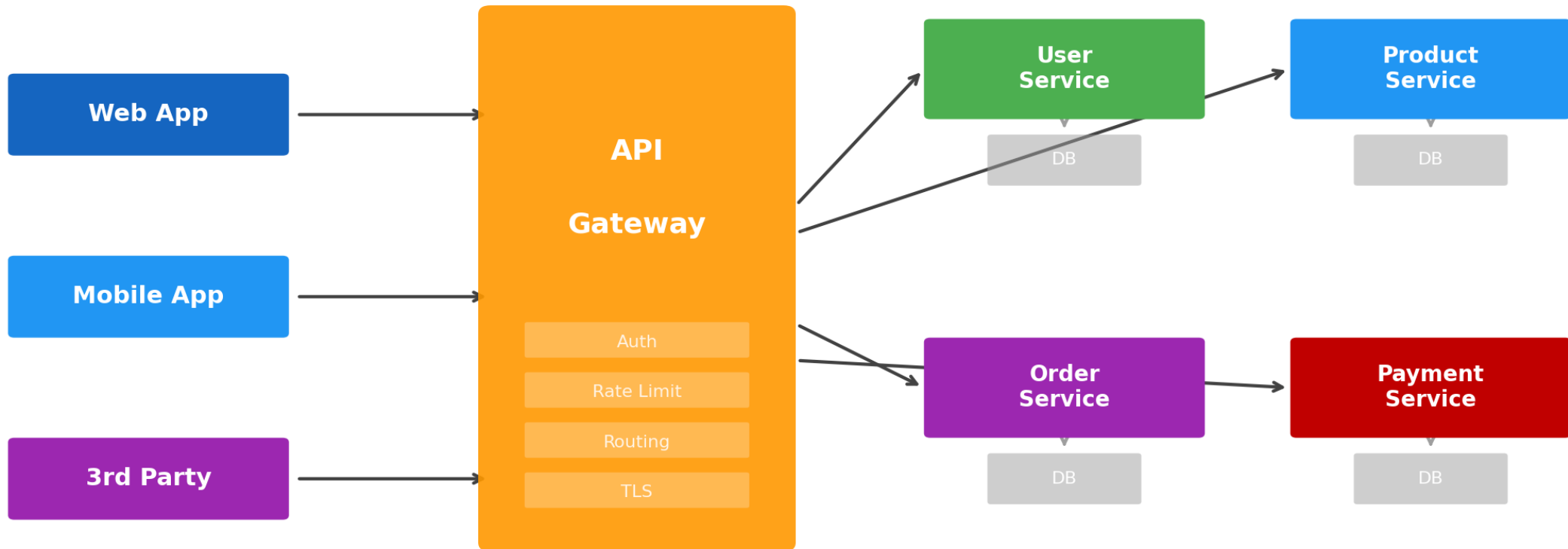
Client-side: the caller queries the registry directly. Server-side: a proxy handles the lookup. Kubernetes uses server-side by default.



API Gateway Pattern

Single entry point for all clients. Handles auth, rate limiting, TLS, and routing to backend services. Each service owns its own database.

API Gateway Pattern



Common Architecture Patterns

Production systems use patterns to manage traffic flow and isolate concerns. These four appear in nearly every large-scale deployment.

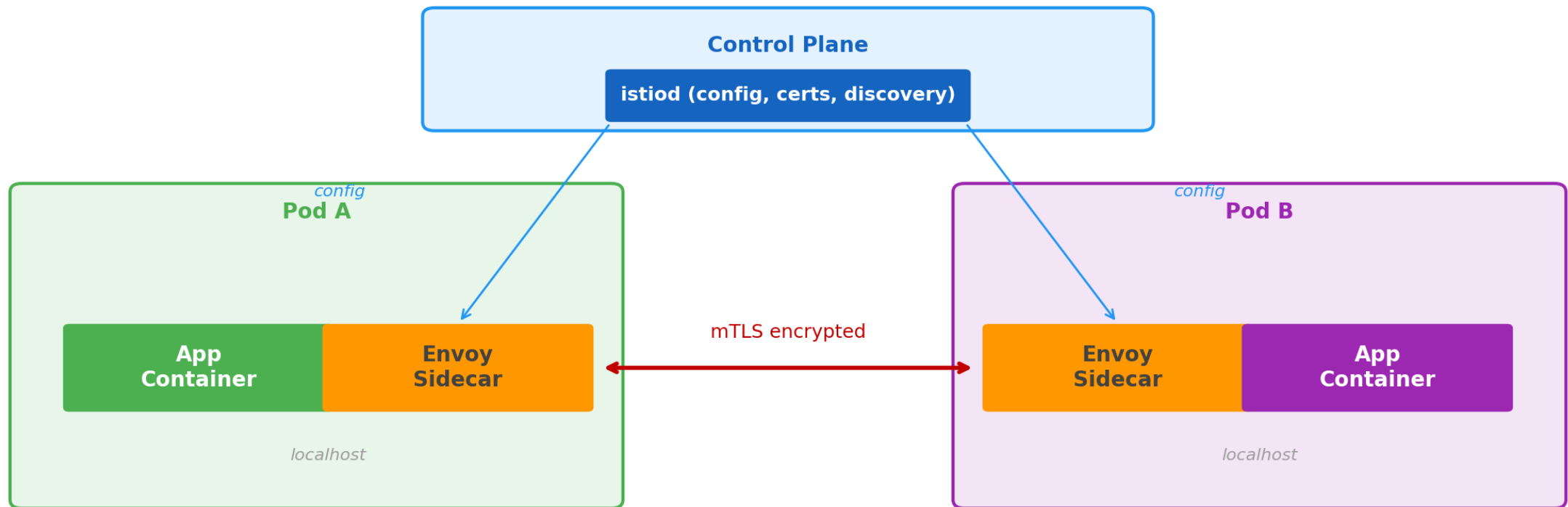
- API Gateway (Kong, Ambassador): single entry point for auth, rate limiting, routing
- Backend for Frontend: Spotify runs separate BFFs for mobile, web, and embedded devices
- Sidecar/Mesh: Envoy handles mTLS, retries, observability. Tradeoff: 2-5ms latency, 30-50MB RAM per sidecar
- Strangler Fig: Shopify migrated from Rails by routing through a facade over 3+ years



Sidecar Pattern / Service Mesh

Envoy sidecars handle mTLS, retries, circuit breaking, and telemetry. The control plane (istiod) manages all proxy configuration centrally.

Sidecar Pattern / Service Mesh (Istio + Envoy)



Sidecar handles: mTLS • Retries • Circuit Breaking • Load Balancing • Metrics • Tracing

Deploying and Managing Microservices

Independent deployment is the whole point. If shipping one service requires coordinating five teams, you have a distributed monolith.

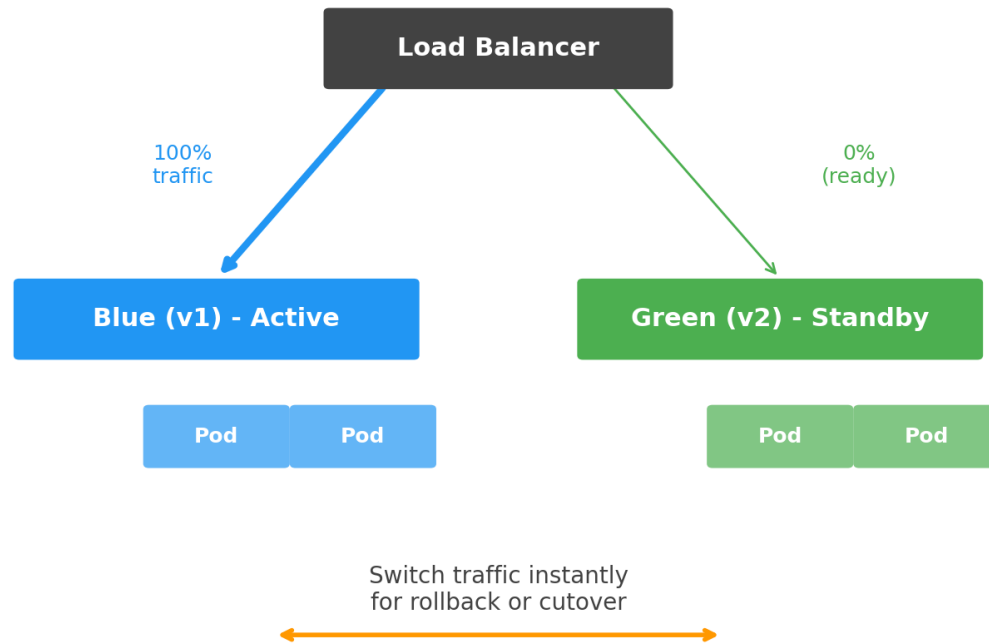
- Each service gets its own CI/CD pipeline, Dockerfile, and Helm chart. At 50 services, that is 50 pipelines to maintain
- Canary deploys (Argo Rollouts, Flagger) send 5% of traffic to new versions, auto-rollback if error rates spike
- Probes are non-negotiable: liveness (restart if stuck), readiness (remove from LB), startup (grace period for slow JVMs)
- Resource limits prevent noisy neighbors. One memory leak should not OOM-kill pods on the same node



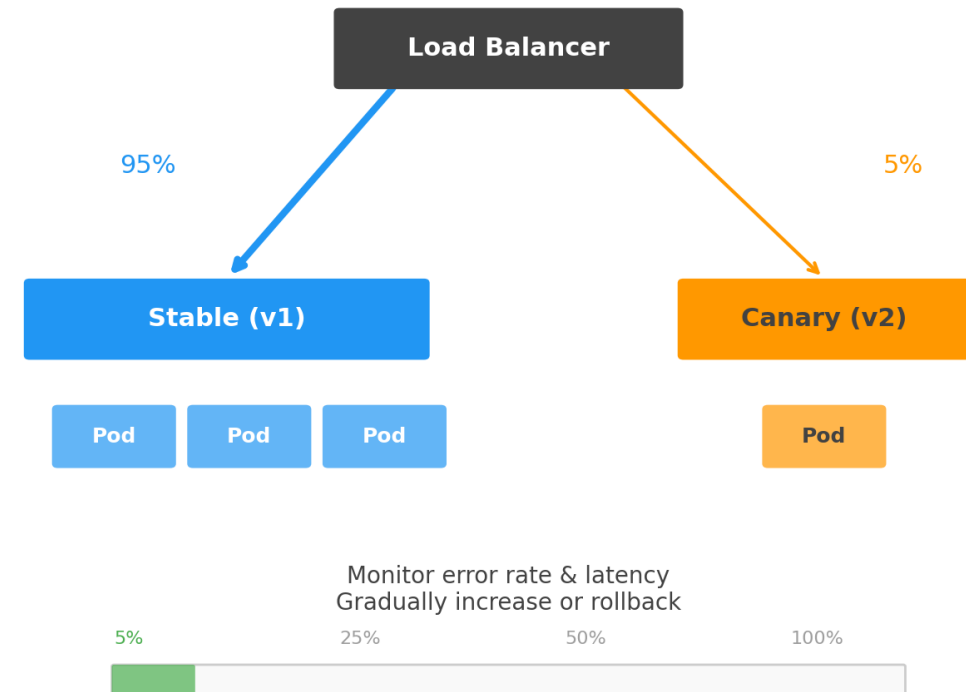
Deployment Strategies

Blue-green: instant switchover between two identical environments. Canary: gradual traffic shift with automated rollback.

Blue-Green Deployment



Canary Deployment



Labs: Deploying Microservices



ConfigMaps and Secrets



ConfigMap

ConfigMap

- Many applications require configuration via:
 - Config Files
 - Command-Line Arguments
 - Environment Variables
- These need to be decoupled from images to keep portable
- ConfigMap API provides mechanisms to inject containers with configuration data
- Store individual properties or entire config files/JSON blobs
- Key-Value Pairs

ConfigMap

- Not meant for sensitive information
- PODs or controllers can use ConfigMaps

1. Populate the value of environment variables
2. Set command-line arguments in a container
3. Populate config files in a volume

```
kind: ConfigMap
apiVersion: v1
metadata:
  creationTimestamp: 2016-02-18T19:14:38Z
  name: example-config
  namespace: default
data:
  example.property.1: hello
  example.property.2: world
  example.property.file: |-
    property.1=value-1
    property.2=value-2
    property.3=value-3
```

ConfigMap from directory

- 2 files in docs/user-guide/configmap/kubectl
 - game.properties
 - ui.properties

game.properties

```
enemies=aliens  
lives=3  
enemies.cheat=true  
enemies.cheat.level=noGoodRotten  
secret.code.passphrase=UUDDLRLRBABAS  
secret.code.allowed=true  
secret.code.lives=30
```

ui.properties

```
color.good=purple  
color.bad=yellow  
allow.textmode=true  
how.nice.to.look=fairlyNice
```

ConfigMap from directory

```
kubectl create configmap game-config --from-file=docs/user-guide/configmap/kubectl
```

```
kubectl describe configmaps game-config
```

```
Name: game-config
```

```
Namespace: default
```

```
Labels: <none>
```

```
Annotations: <none>
```

```
Data
```

```
====
```

```
game.properties: 121 bytes
```

```
ui.properties: 83 bytes
```

ConfigMap from directory

```
kubectl get configmaps game-config -o yaml
```

```
apiVersion: v1
data:
  game.properties: |-
    enemies=aliens
    lives=3
    ...
  ui.properties: |
    color.good=purple
    ...
kind: ConfigMap
metadata:
  creationTimestamp: 2016-02-18T18:34:05Z
  name: game-config
  namespace: default
  resourceVersion: "407"-
  selfLink: /api/v1/namespaces/default/configmaps/game-config
  uid: 30944725-d66e-11e5-8cd0-68f728db1985
```

ConfigMap from files

```
kubectl get configmaps \  
game-config-2 \  
-o yaml
```

```
kubectl create configmap \  
game-config-2 \  
--from-file=file1 \  
--from-file=file2
```

```
apiVersion: v1  
data:  
  game.properties: |-  
    enemies=aliens  
    lives=3  
    ...  
  ui.properties: |  
    color.good=purple  
    ...  
kind: ConfigMap  
metadata:  
  creationTimestamp: 2016-02-18T18:52:05Z  
  name: game-config-2  
  namespace: default  
  resourceVersion: "516"  
  selfLink: /api/v1/namespaces/default/configmaps/game-config-2  
  uid: b4952dc3-d670-11e5-8cd0-68f728db1985
```

ConfigMap options

`--from-file=/path/to/directory`

`--from-file=/path/to/file1 (/path/to/file2)`

Literal key=value: `--from-literal=special.how=very`

ConfigMap in PODs

■ Populate Environment Variables

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: special-config
  namespace: default
data:
  special.how: very
  special.type: charm
```

OUTPUT

```
SPECIAL_LEVEL_KEY=very
```

```
apiVersion: v1
kind: Pod
metadata:
  name: dapi-test-pod
spec:
  containers:
    - name: test-container
      image: busybox
      command: ["/bin/sh", "-c", "env"]
      env:
        - name: SPECIAL_LEVEL_KEY
          valueFrom:
            configMapKeyRef:
              name: special-config
              key: special.how
  restartPolicy: Never
```

ConfigMap Restrictions

- ConfigMaps must be created before they are consumed
- ConfigMaps can only be referenced by objects in the same namespace
- Quota for ConfigMap size not implemented yet
- Can only use ConfigMap for PODs created through API server.

Secrets



What secrets do applications have?

- Database credentials
- API credentials & endpoints (Twitter, Facebook etc.)
- Infrastructure API credentials (Google, Azure, AWS)
- Private keys (TLS, SSH)
- Many more!

Application Secrets

It is a bad idea to include these secrets in your code.

- Accidentally push up to GitHub with your code
- Push into your file storage and forget about
- Etc.

Application Secrets

There are bots crawling GitHub searching for secrets

Real life example:

Dev put keys out on GitHub, woke up next morning with a ton of emails and missed calls from Amazon

- 140 instances running under Dev's account.
- \$2,375 worth of Bitcoin mining

Create Secret

- Designed to hold all kinds of sensitive information
- Can be used by Pods (filesystem & environment variables) and the underlying kubelet when pulling images

```
apiVersion: v1
kind: Secret
metadata:
  name: mysecret
Type: Opaque
data:
  password: mmyWfoidfluL==
  username: NyhdOKwB
```

Pod Secret

```
kind: Pod
metadata:
  name: secret-env-pod
spec:
  containers:
  - name: mycontainer
    image: redis
    env:
    - name: SECRET_USERNAME
      valueFrom:
        secretKeyRef:
          name: mysecret
          key: username
```

Volume Secret

```
spec:
  containers
  - name: mycontainer
    image: redis
    volumeMounts:
    - name: "secrets"
      mountPath: "/etc/my-secrets"
      readOnly: true
  volumes:
  - name: "secrets"
    secret:
      secretName: "mysecret"
```

Labs: ConfigMap & Secrets

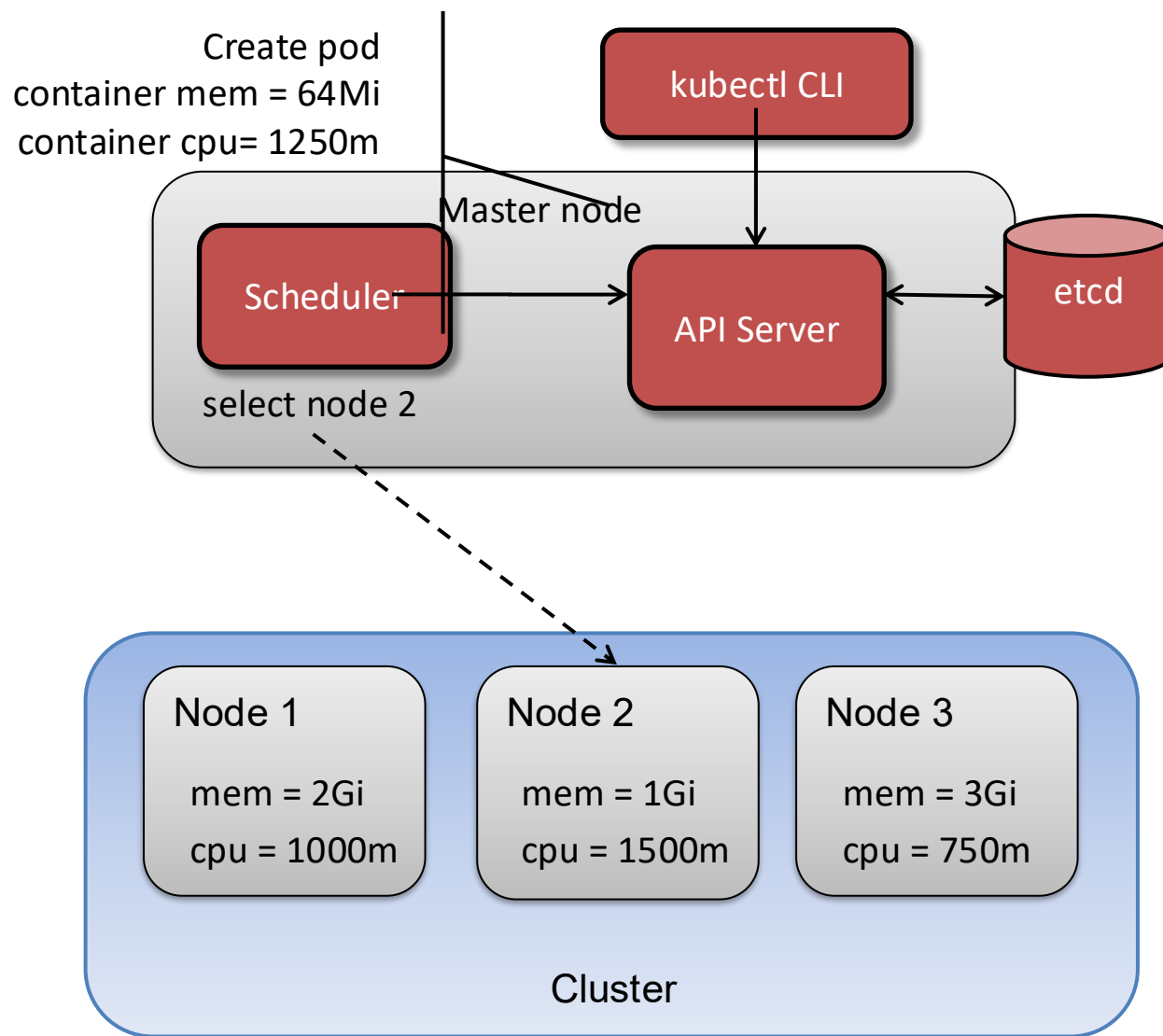


Pod Scheduling



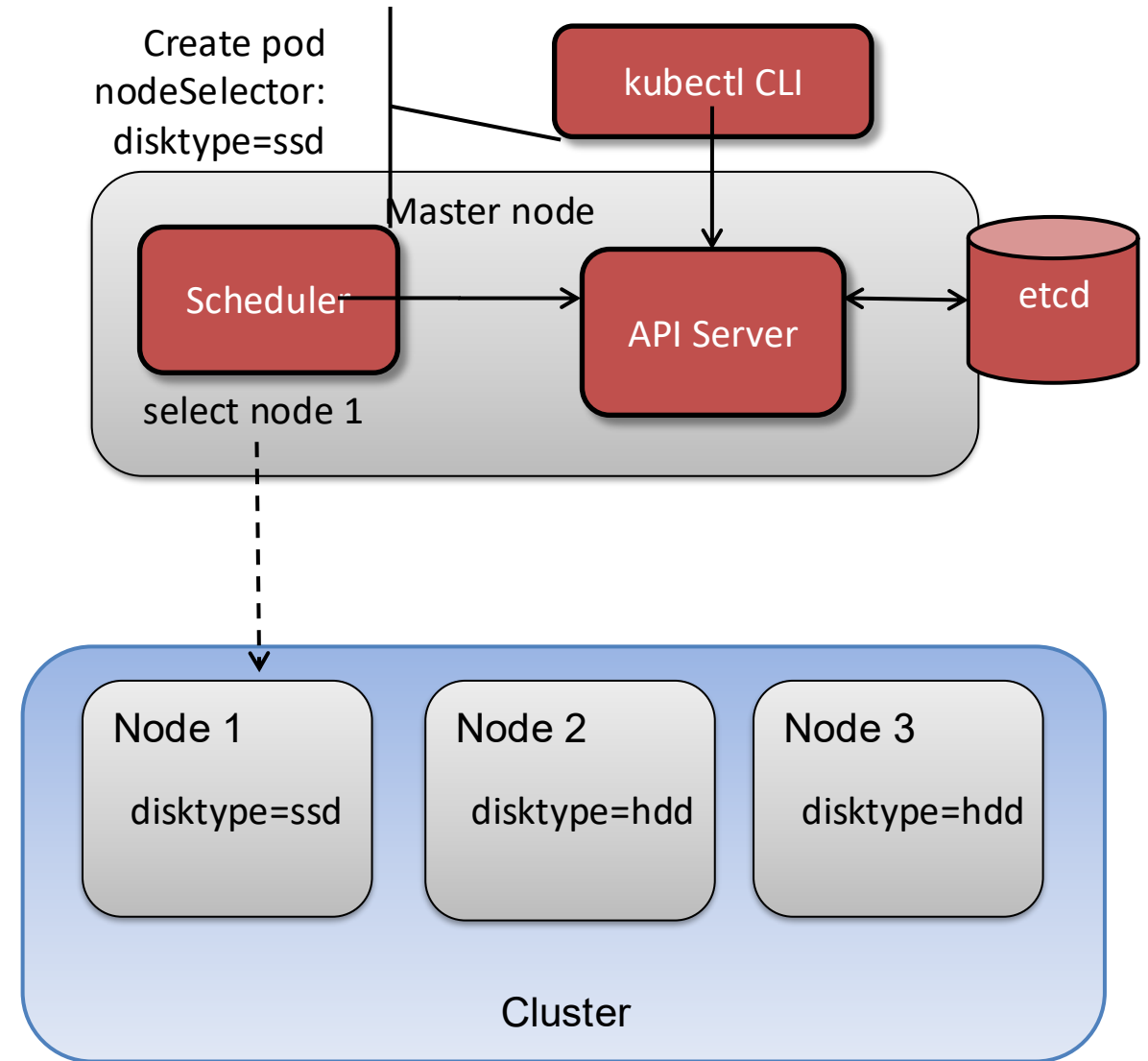
Pod Scheduling Overview

- kube-scheduler on the master node assigns new pod requests to appropriate worker nodes
- Default scheduler takes account of
 - Available node CPU/RAM
 - Resource requests from new pod – sum of resource requests of pod containers
- Default scheduler will automatically
 - Schedule pod on node with sufficient free resources
 - Spread pods across nodes in the cluster
- Can specify custom schedulers for pods



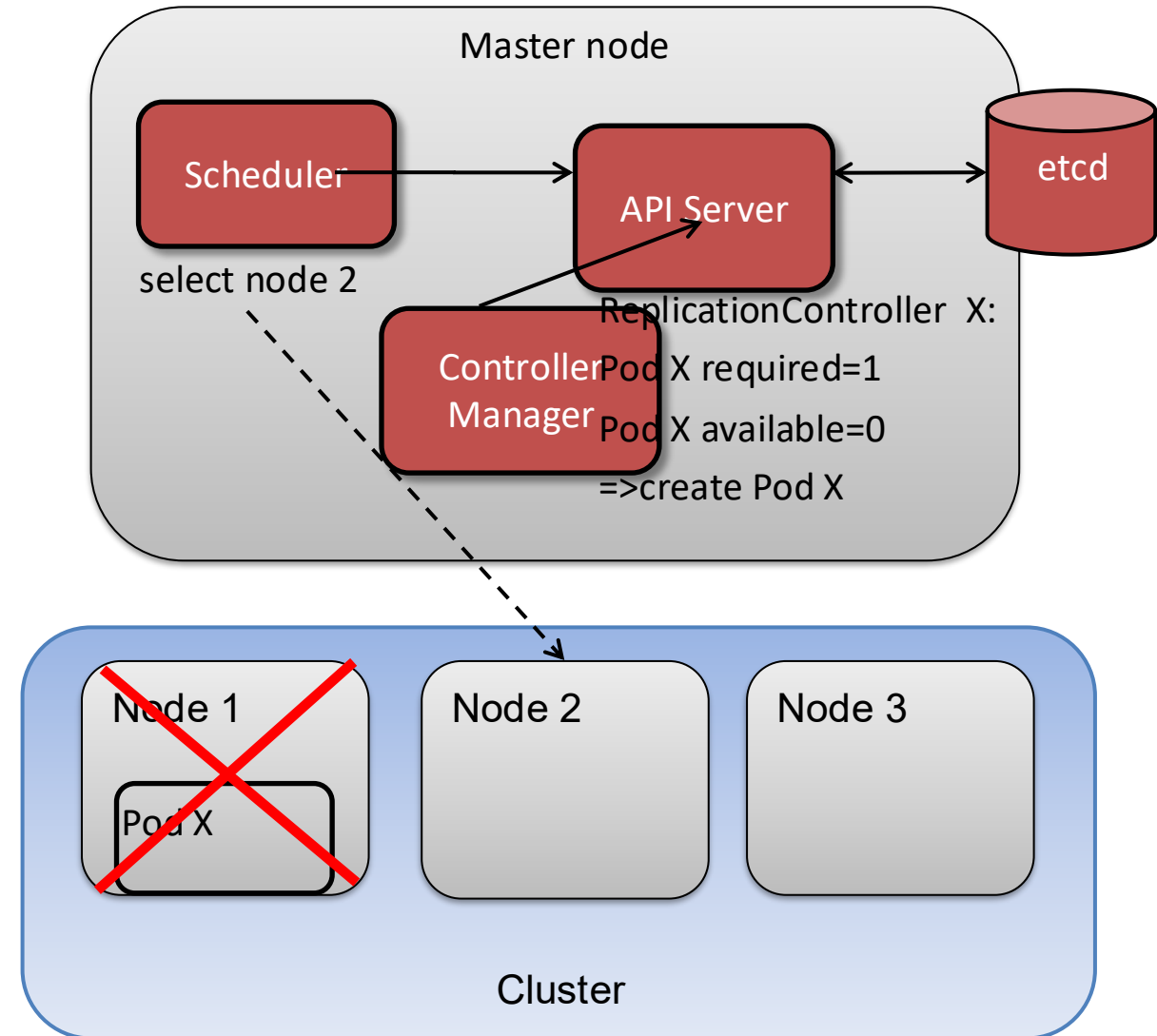
Pod Scheduling with Default Scheduler

- Nodes carry labels indicating topology and other resource notes
- Users can require pods to be scheduled on nodes with specific label(s) via a nodeSelector in container spec
- K8s 1.6 adds more advanced scheduling options, including
 - Node affinity/anti-affinity generalizing the nodeSelector feature
 - Node taints that prevent scheduling of pods to nodes unless pod 'tolerates' the taint
 - Pod affinity/anti-affinity to control relative placement of pods



Pod Re-creation Driven by Control Loops

- kube-scheduler performs same node selection operation when new pod created due to e.g. node loss
- kube-controller-manager runs controllers like ReplicationController managing number of pod instances available
- kube-controller-manager will initiate request for new pods as needed, which will be scheduled by kube-scheduler per pod/container spec



Kubernetes Scheduling

Node Selector

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx
  labels:
    env: test
spec:
  container:
  - name: nginx
    image: nginx
    nodeSelector:
      disktype: ssd
```

Built-in Node Labels

- `kubernetes.io/hostname`
- `failure-domain.beta.kubernetes.io/zone`
- `failure-domain.beta.kubernetes.io/region`
- `beta.kubernetes.io/instance-type`
- `beta.kubernetes.io/os`
- `beta.kubernetes.io/arch`
- `node.openshift.io/os_id`

Kubernetes Advanced POD Scheduling



Kubernetes Scheduling

- Scheduler Profiles
- Node Selector
- Node Affinity
- Pod Affinity
- Pod Anti Affinity
- 3rd party schedulers

Kubernetes Scheduling

- Node affinity/anti-affinity
 - generalizing the nodeSelector feature
- Node taints
 - prevent scheduling of pods to nodes unless pod 'tolerates' the taint
- Pod affinity/anti-affinity
 - control relative placement of pods

Node Affinity

The affinity/anti-affinity feature greatly expands the types of constraints you can express.

The key enhancements are:

- More expressive language (not just “AND of exact match”)
- Rule is “soft”/”preference” so if it doesn’t match Pods are still scheduled.

Kubernetes Scheduling

Node Affinity

```
affinity:  
  nodeAffinity:  
    requiredDuringSchedulingIgnoredDuringExecution:  
      nodeSelectorTerms:  
        - matchExpressions:  
          - key: "failure-domain.beta.kubernetes.io/zone"  
            operator: In  
            values: ["us-central1-a"]
```

Kubernetes Scheduling

Node Affinity

```
affinity:  
  nodeAffinity:  
    preferredDuringSchedulingIgnoredDuringExecution:  
      nodeSelectorTerms:  
        - matchExpressions:  
          - key: "failure-domain.beta.kubernetes.io/zone"  
            operator: In  
            values: ["us-central1-a"]
```

Kubernetes Scheduling

Weight example

```
affinity:
  nodeAffinity:
    preferredDuringSchedulingIgnoredDuringExecution:
      - weight: 5
        preference:
          - matchExpressions:
              - key: env
                operator: In
                values:
                  - dev
            preferredDuringSchedulingIgnoredDuringExecution:
              - weight: 1
                preference:
                  - matchExpressions:
                      - key: team
                        operator: In
                        values:
                          - engineering
```

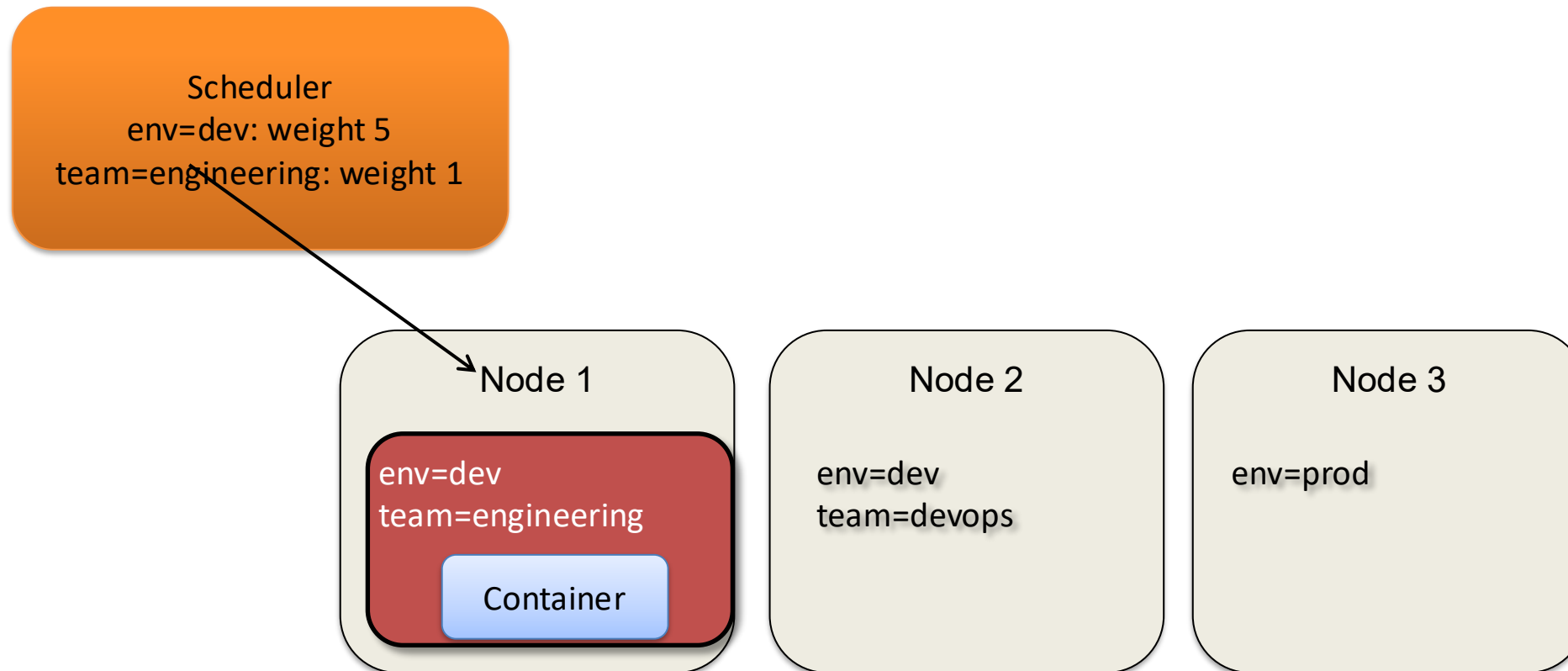
Node Affinity

Kubernetes supports "weighted scheduling"

- The higher this weighting the more weight is given to that rule.
- Scoring is done by summarizing weighting per node
 - 2 rules weights 1 and 5
 - If both match = score of 6
 - If only 1 rule matches = score of 1
- Node with highest total weight score receives Pod.

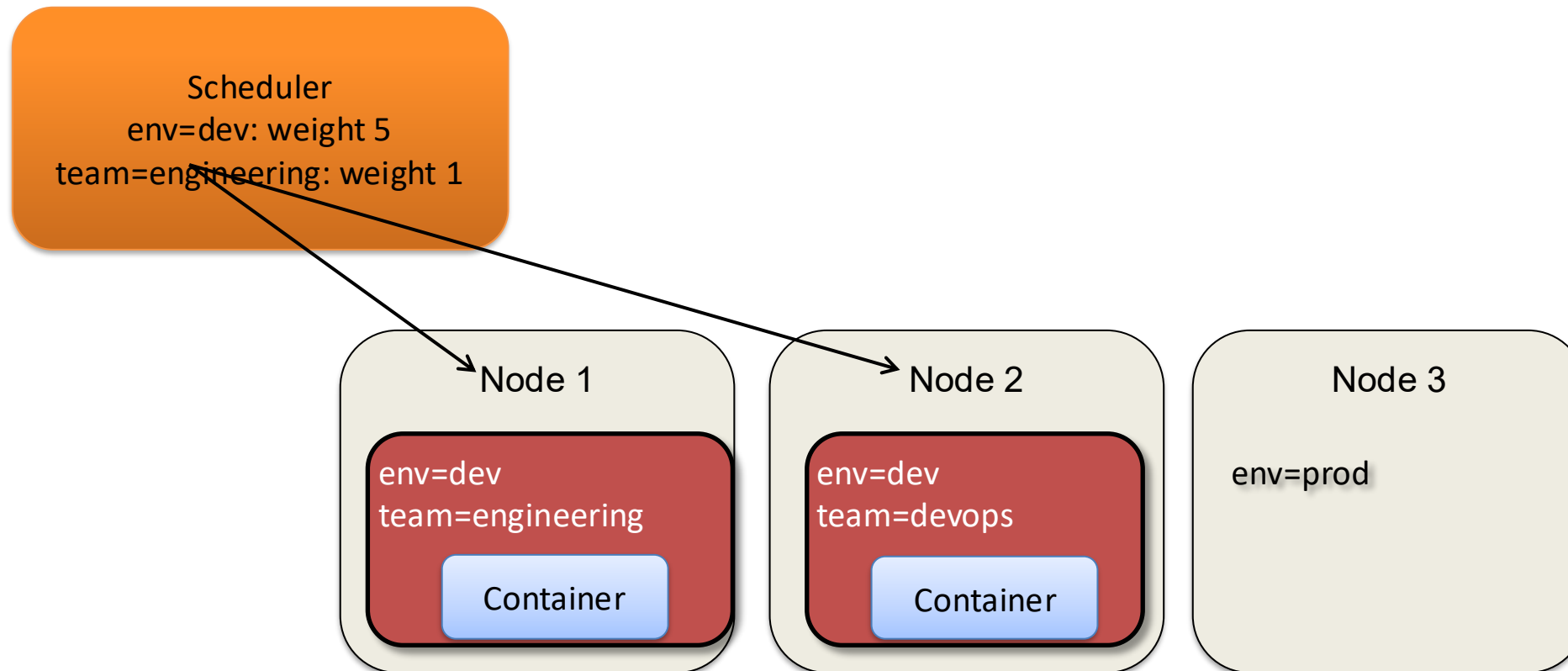
Node Affinity: Weight

- Schedule Pods onto nodes that have labels
 - env = dev
 - team = engineering



Node Affinity: Weight

- Schedule Pods onto nodes that have labels
 - env = dev
 - team = engineering



Kubernetes Scheduling

Node Anti-Affinity

```
affinity:  
  nodeAffinity:  
    requiredDuringSchedulingIgnoredDuringExecution:  
      nodeSelectorTerms:  
        - matchExpressions:  
          - key: "failure-domain.beta.kubernetes.io/zone"  
            operator: NotIn  
            values: ["us-central1-a"]
```

Operators

Valid operators

- In
- NotIn
- Exists
- DoesNotExist
- Gt (Greater Than)
- Lt (Less Than)

Node Taints

- Allows you to mark (“taint”) a node
 - No pods can be scheduled onto tainted nodes
 - Useful when all or most PODs should not be scheduled on a node
 - Mark your master node as schedulable only by Kubernetes system components
 - Keep regular pods away from nodes that have special hardware so as to leave room for pods that need the special hardware.

Node Tolerations

- To allow a Pod to be scheduled onto a 'tainted' node it must have:

```
tolerations:  
- key: "key"  
  operator: "Equal"  
  value: "value"  
  effect: "NoSchedule"
```

Pod Affinity

- Define how pods should be placed relative to one another
- Spread or pack pods within a service or relative to pods in other services

```
affinity:  
  podAffinity:  
    requiredDuringSchedulingIgnoredDuringExecution:  
      - labelSelector:  
          matchExpressions:  
            - key: service  
              operator: In  
              values: ["S1"]  
        topologyKey: failure-domain.beta.kubernetes.io/zone
```

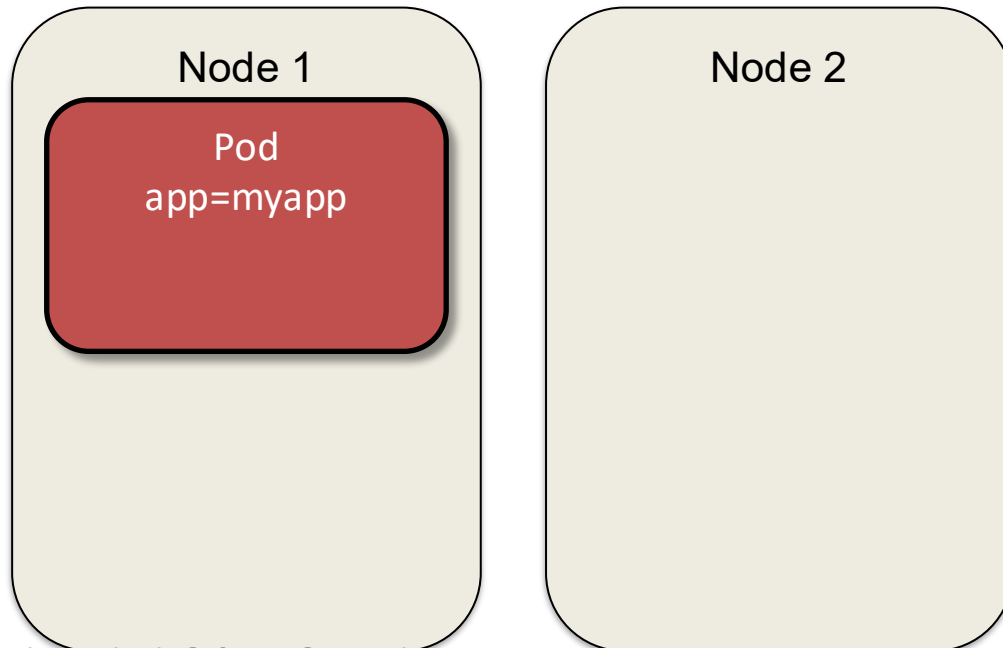
Pod Affinity

- Define how pods should be placed relative to one another
- Spread or pack pods within a service or relative to pods in other services

```
affinity:  
  podAffinity:  
    preferredDuringSchedulingIgnoredDuringExecution:  
      - labelSelector:  
          matchExpressions:  
            - key: service  
              operator: In  
              values: ["S1"]  
        topologyKey: failure-domain.beta.kubernetes.io/zone
```

Pod Affinity

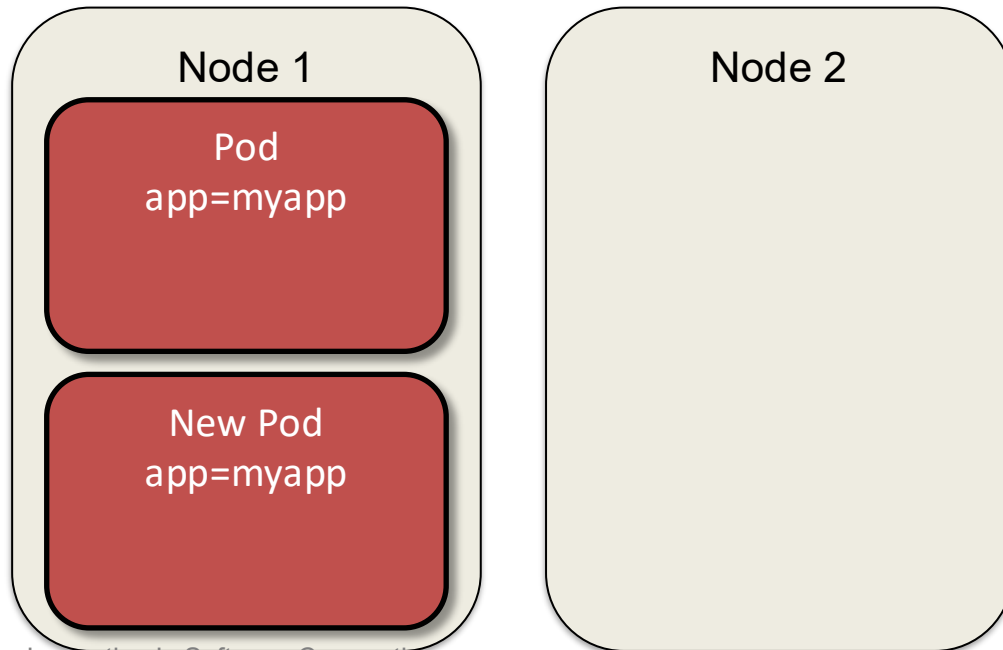
- Schedule Pods onto nodes that have same labels
 - app=myapp
 - operator: In
 - kubernetes.io/hostname



```
affinity:
  podAffinity:
    requiredDuringSchedulingIgnoredDuringExecution:
      - labelSelector:
          matchExpressions:
            - key: "app"
              operator: In
              values:
                - myapp
        topologyKey: "kubernetes.io/hostname"
```

Pod Affinity

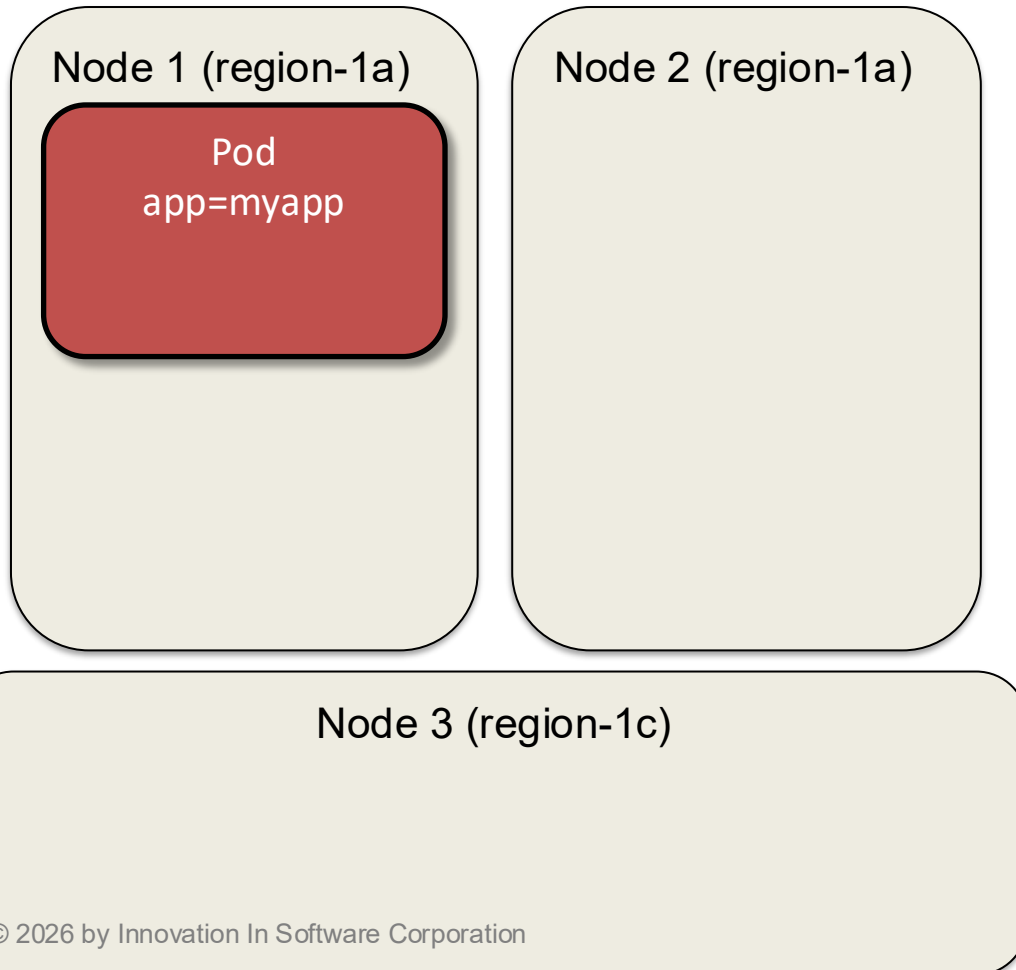
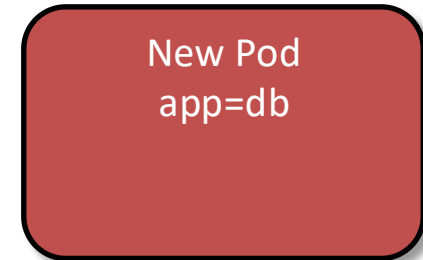
- Schedule Pods onto nodes that have same labels
 - app=myapp
 - operator: In
 - kubernetes.io/hostname



```
affinity:
  podAffinity:
    requiredDuringSchedulingIgnoredDuringExecution:
      - labelSelector:
          matchExpressions:
            - key: "app"
              operator: In
              values:
                - myapp
        topologyKey: "kubernetes.io/hostname"
```

Pod Affinity

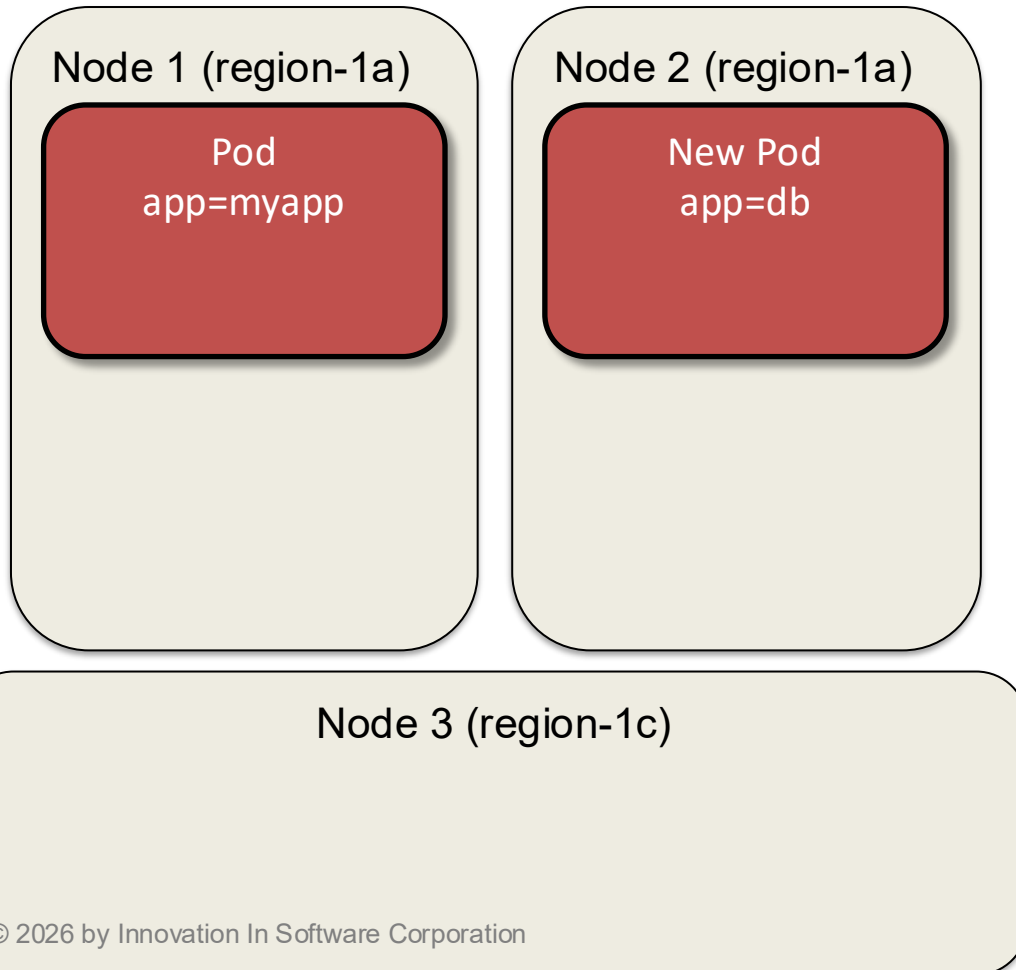
- Schedule Pods onto nodes in same region
 - app=db
 - failure-domain.beta.kubernetes.io/zone



```
affinity:
  podAffinity:
    requiredDuringSchedulingIgnoredDuringExecution:
      - labelSelector:
          matchExpressions:
            - key: "app"
              operator: In
              values:
                - myapp
        topologyKey: "failure-domain.beta.kubernetes.io/zone"
```

Pod Affinity

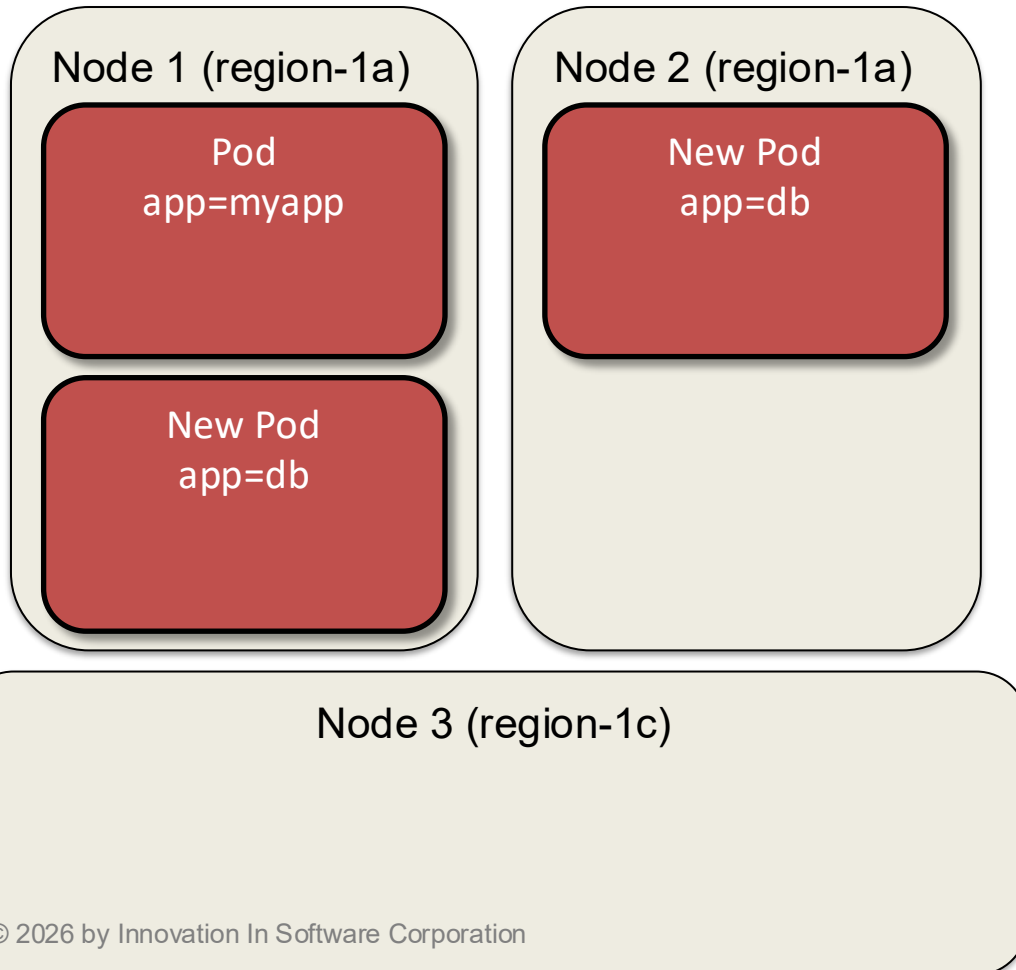
- Schedule Pods onto nodes in same region
 - app=db
 - failure-domain.beta.kubernetes.io/zone



```
affinity:  
  podAffinity:  
    requiredDuringSchedulingIgnoredDuringExecution:  
      - labelSelector:  
          matchExpressions:  
            - key: "app"  
              operator: In  
              values:  
                - myapp  
        topologyKey: "failure-domain.beta.kubernetes.io/zone"
```

Pod Affinity

- Schedule Pods onto nodes in same region
 - app=db
 - failure-domain.beta.kubernetes.io/zone



```
affinity:
  podAffinity:
    requiredDuringSchedulingIgnoredDuringExecution:
      - labelSelector:
          matchExpressions:
            - key: "app"
              operator: In
              values:
                - myapp
        topologyKey: " failure-domain.beta.kubernetes.io/zone"
```

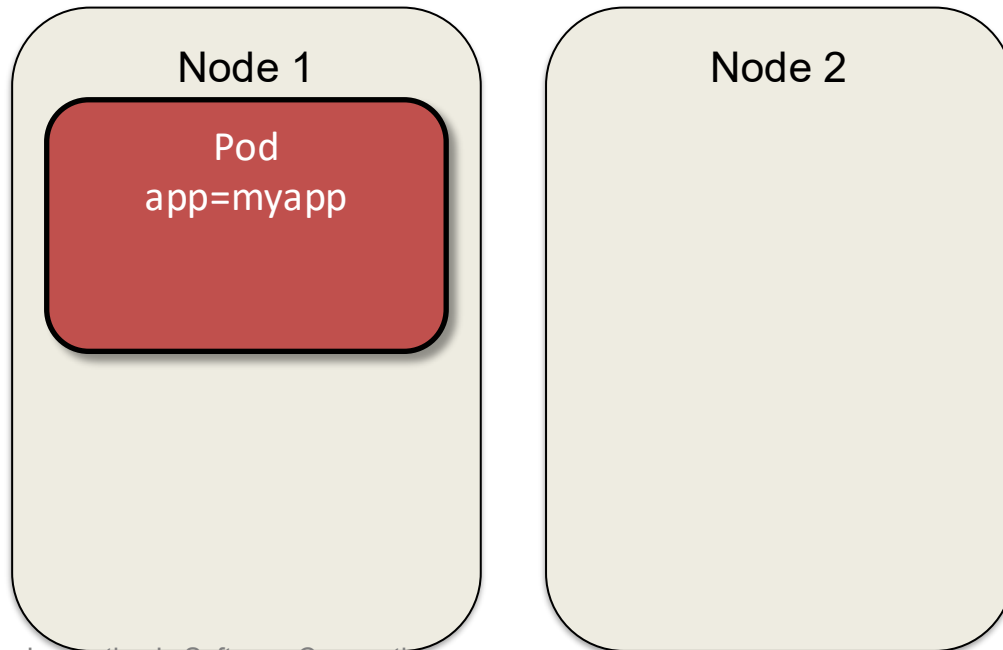
Pod Anti-Affinity

- Define how pods should be placed relative to one another
- PODs of different services run on different nodes.

```
affinity:  
  podAntiAffinity:  
    preferredDuringSchedulingIgnoredDuringExecution:  
      - labelSelector:  
          matchExpressions:  
            - key: service  
              operator: In  
              values: ["S1"]  
        topologyKey: kubernetes.io/hostname
```

Pod Anti-Affinity

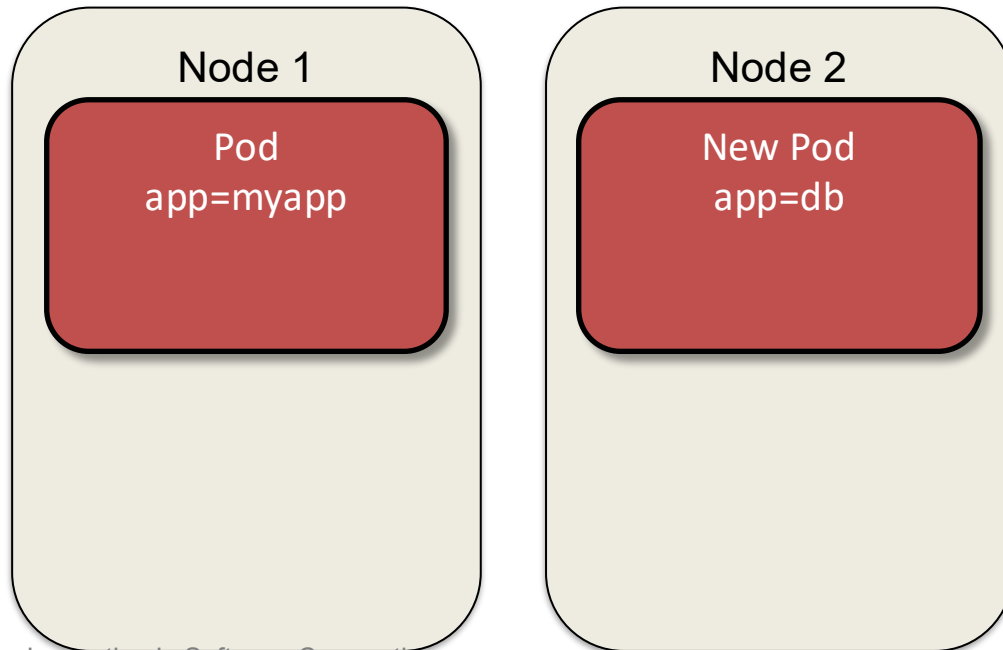
- Do not schedule Pods onto nodes with matching labels.
 - app=db
 - operator: In
 - kubernetes.io/hostname



```
affinity:
  podAntiAffinity:
    requiredDuringSchedulingIgnoredDuringExecution:
      - labelSelector:
          matchExpressions:
            - key: "app"
              operator: In
              values:
                - myapp
        topologyKey: "kubernetes.io/hostname"
```

Pod Anti-Affinity

- Do not schedule Pods onto nodes with matching labels.
 - app=db
 - operator: In
 - kubernetes.io/hostname



```
affinity:  
  podAntiAffinity:  
    requiredDuringSchedulingIgnoredDuringExecution:  
      - labelSelector:  
          matchExpressions:  
            - key: "app"  
              operator: In  
              values:  
                - myapp  
        topologyKey: "kubernetes.io/hostname"
```

Labs: OpenShift Scheduling



Questions

